BCS402_Module 5

RN SHETTY TRUST®

RNS INSTITUTE OF TECHNOLOGY

Autonomous Institution Affiliated to Visvesvaraya Technological University, Belagavi
Approved by AICTE, New Delhi, Accredited by NAAC with 'A+' Grade
Channasandra, Dr. Vishnuvardhan Road, Bengaluru - 560 098
Ph: (080) 28611880 / 1; www.rnsit.ac.in

DEPARTMENT OF CSE (AI & ML)

Database Management Systems

BCS402

Module 5

Transaction Processing, Concurrency Control in Databases, Database Recovery Techniques

MODULE: 5

CONTENTS:

- Transaction Processing:
- Concurrency Control in Databases
- Database Recovery Techniques

Transaction processing

- Introduction to Transaction Processing
- Transaction and System concepts,
- Desirable properties of Transactions,
- Characterizing schedules based on recoverability,
- Characterizing schedules based on Serializability,
- Transaction support in SQL.

Concurrency Control in Databases

Database Recovery Techniques

Textbook : Fundamentals of Database Systems, Ramez Elmasri & Shamkant B. Navathe, 7th Edition, Pearson.

Chapter Ch 20.1 to 20.6, Ch 21.1 to 21.2, Ch 22.1 – 22.5, 22.7

5.1 Introduction to Transaction Processing

A Transaction Processing System (TPS**)** is a component of a database management system that manages the execution of database transactions in a reliable, consistent, and efficient manner. A transaction is a sequence of database operations (such as read and write) that performs a single logical unit of work, for example, transferring money between bank accounts or placing an order in an online system.

In a multi-user environment, several transactions may execute at the same time. This is known as concurrent execution of transactions. While concurrency improves system performance and resource utilization, it may lead to problems such as lost updates, dirty reads, and inconsistent data if not properly controlled. Therefore, concurrency control techniques are required to ensure that the outcome of executing transactions concurrently is the same as if they were executed one after another.

In addition, transactions may fail due to system crashes, power failures, or logical errors. To maintain database correctness, the system must be able to **recover** from such failures. Recovery techniques ensure that the database is restored to a consistent state by undoing incomplete transactions and redoing committed ones.

Thus, a Transaction Processing System provides mechanisms for concurrency control and recovery, ensuring the properties of atomicity, consistency, isolation, and durability (ACID), which are essential for reliable database operation.

5.1.1 Single-User versus Multiuser Systems

- A DBMS can be classified as:
  - **Single-user DBMS**: Only one user can access the system at a time.
  - **Multiuser DBMS**: Many users can access the database concurrently.
- Single-user DBMSs are mainly used on personal computers.
- Most real-world database systems are multiuser systems.
- Examples of multiuser database systems:
  - Airline reservation systems
  - Banking systems
  - Insurance systems
  - Stock exchange systems
  - Supermarket systems
- In these systems, **hundreds or thousands of users** may submit transactions concurrently.

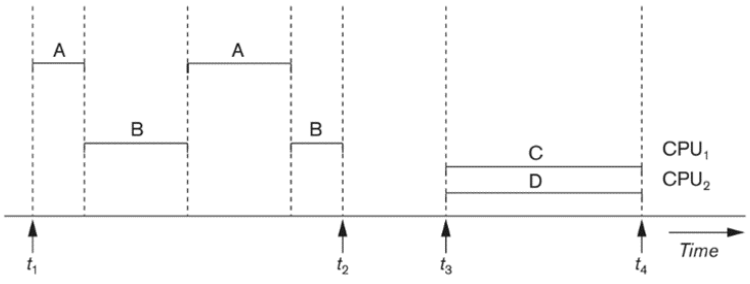


Figure 20.1 Interleaved processing versus parallel processing of concurrent transactions.

Role of Multiprogramming

- Multiple users can access databases simultaneously because of multiprogramming.
- Multiprogramming allows the operating system to execute multiple processes at the same time.

- A CPU can execute only one process at a time.
- The operating system:
  - Executes some commands of one process,
  - Suspends that process,
  - Executes commands of another process,
  - Then resumes the first process later.
- This interleaved execution of processes is shown in Figure 20.1, where processes A and B are executed in an interleaved manner.

Advantages of Interleaving

- Interleaving keeps the CPU busy when one process is waiting for I/O operations.
- The CPU switches to another process instead of remaining idle during I/O time, as illustrated in Figure 20.1.
- Interleaving also prevents a long process from delaying other processes.
- Thus, interleaved execution shown in Figure 20.1 improves system efficiency and response time.

Parallel Processing

- If a computer system has multiple CPUs**,** true parallel processing is possible.
- Multiple processes can run at the same time on different CPUs.
- This is illustrated in Figure 20.1 by processes C and D, which execute in parallel.
- Most concurrency control concepts in databases assume interleaved execution, as shown for processes A and B in Figure 20.1.

5.1.2 Transactions, Database Items, Read and Write Operations, and DBMS Buffers

- A transaction is an executing program that forms a logical unit of database processing.
- A transaction consists of one or more database operations such as:
  - Insertion
  - Deletion

- Update (modification)
- Retrieval

- Database operations can be:
  - Embedded in an application program, or
  - Issued interactively using SQL.
- Transaction boundaries can be defined using:
  - Begin transaction and End transaction statements.
- A single program may contain multiple transactions.
- Types of transactions:
  - **Read-only transaction**: only retrieves data.
  - **Read-write transaction**: retrieves and updates data.

Database Items and Granularity

- A database is modeled as a collection of **named data items.**
- The size of a data item is called its **granularity.**
- A data item can be:
  - A disk block
  - A record
  - A single field (attribute)
- Each data item has a **unique name** (e.g., disk block address or record ID).
- Transaction concepts are **independent of data item size (granularity)**.

Read and Write Operations

- Basic database operations are:
  - **read_item(X)**: reads database item X into program variable X.

- **write_item(X)**: writes program variable X into database item X.

Steps in **read_item(X)**:

- Locate disk block containing X.
- Copy disk block to main memory buffer (if not already present).
- Copy item X to program variable X.

Steps in **write_item(X)**:

- Locate disk block containing X.
- Copy disk block to buffer (if not already present).
- Copy program variable X into buffer.
- Write updated block back to disk (immediately or later).

Actual database update happens when the buffer is written back to disk (step 4).

DBMS Buffers and Buffer Replacement

- DBMS maintains a **database cache (buffers)** in main memory.
- Each buffer holds one **disk block.**
- When buffers are full, **a buffer replacement policy** is used.
- Common policy:
  - **LRU (Least Recently Used)**
- If a replaced buffer was modified, it must be **written back to disk.**

Read-set and Write-set

- A transaction has:
  - **Read-set**: items it reads.
  - **Write-set**: items it writes.
- Example:

- If a transaction reads and writes X and Y,
  - Read-set = {X, Y}
  - Write-set = {X, Y}

Importance for Concurrency and Recovery

- Multiple transactions may execute **concurrently.**
- They may access the **same database items.**
- Uncontrolled concurrent execution can cause:
  - Inconsistent database
  - Data conflicts
- Hence, **concurrency control and recovery mechanisms** are required.

5.1.3 Why Concurrency Control Is Needed

Several problems can occur when concurrent transactions execute in an uncontrolled manner

Example:

We consider an Airline reservation DB. Each records is stored for an airline flight which includes Number of reserved seats among other information.

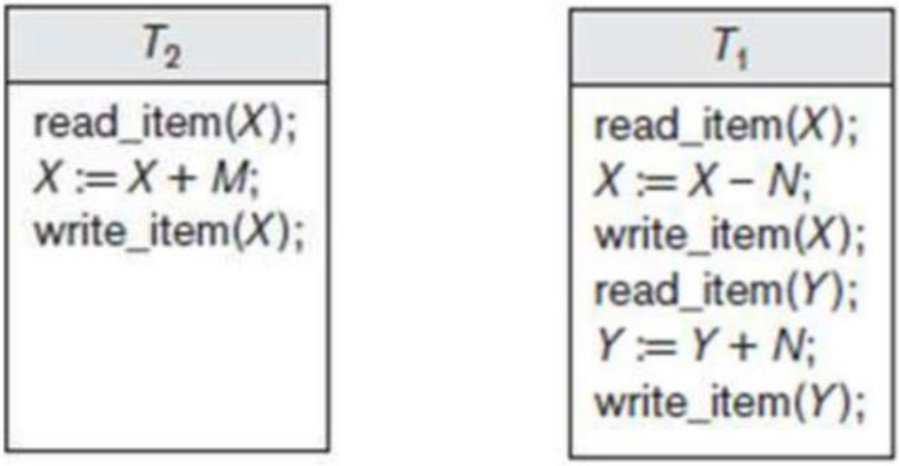
Types of problems we may encounter:

1. The Lost Update Problem

2. The Temporary Update (or Dirty Read) Problem

3. The Incorrect Summary Problem

4. The Unrepeatable Read Problem



Transaction T1

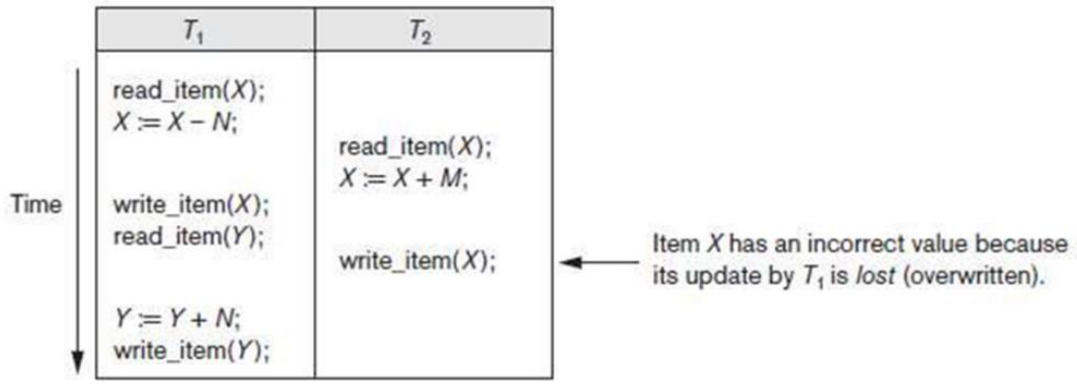
- Transfers N reservations from one flight whose number of reserved seats is stored in the database item named X to another flight whose number of reserved seats is stored in the database item named Y.

Transaction T2

- Reserves M seats on the first flight (X)

1.The Lost Update Problem

Occurs when two transactions that access the same DB items have their operations interleaved in a way that makes the value of some DB item incorrect. Suppose that transactions T1 and T2 are submitted at approximately the same time, and suppose that their operations are interleaved as shown in Figure below



· Final value of item X is incorrect because T2 reads the value of X before T1 changes it in the database, and hence the updated value resulting from T1 is lost.

For example:

X = 80 at the start (there were 80 reservations on the flight)

N = 5 (T1 transfers 5 seat reservations from the flight corresponding to X to the flight corresponding to Y)

M = 4 (T2 reserves 4 seats on X)

The final result should be X = 79.

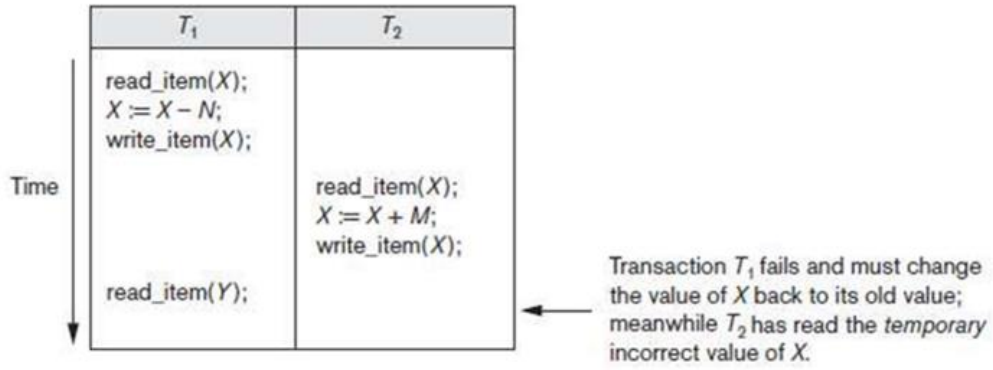
The interleaving of operations shown in Figure is X = 84 because the update in T1 that

removed the five seats from X was lost.

2. The Temporary Update (or Dirty Read) Problem

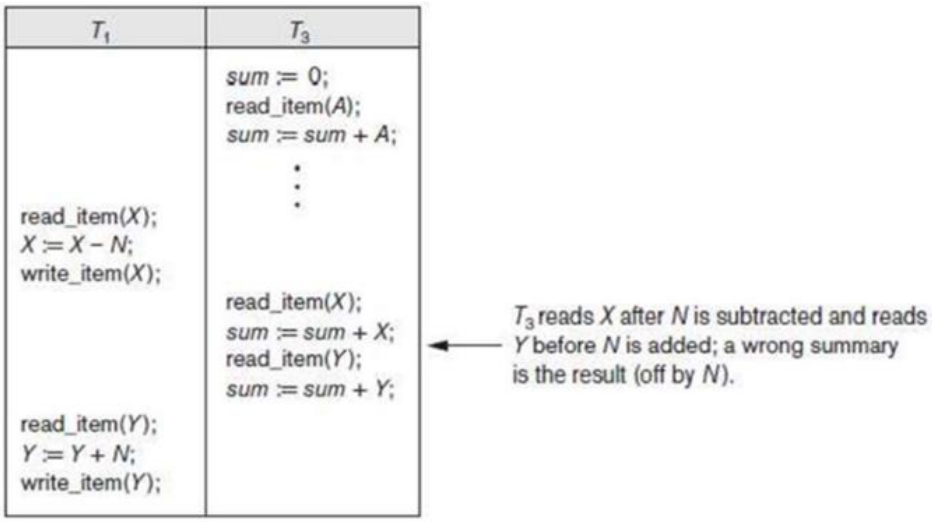
· Occurs when one transaction updates a database item and then the transaction fails for some reason

· Meanwhile the updated item is accessed by another transaction before it is changed back to its original value



3. The Incorrect Summary Problem

If one transaction is calculating an aggregate summary function on a number of db items while other transactions are updating some of these items, the aggregate function may calculate some values before they are updated and others after they are updated.



4. The Unrepeatable Read Problem

Transaction T reads the same item twice and gets different values on each read, since the item was modified by another transaction T` between the two reads. for example, if during an airline reservation transaction, a customer inquires about seat availability on several flights.When the customer decides on a particular flight, the transaction then reads the number of seats on that flight a second time before completing the reservation, and it may end up reading a different value for the item.

5.1.4 Why Recovery Is Needed

Whenever a transaction is submitted to a DBMS for execution, the system is responsible for making sure that either

1. All the operations in the transaction are completed successfully and their effect is recorded permanently in the database or

2. The transaction does not have any effect on the database or any other transactions. In the first case, the transaction is said to be committed, whereas in the second case, the transaction is aborted

If a transaction fails after executing some of its operations but before executing all of them, the operations already executed must be undone and have no lasting effect.

Types of failures

1. A computer failure (system crash):

A hardware, software, or network error occurs in the computer system during transaction execution

Hardware crashes are usually media failures for example, main memory failure.

2. A transaction or system error:

Some operation in the transaction may cause it to fail, such as integer overflow or division by zero

Also occur because of erroneous parameter values

3. Local errors or exception conditions detected by the transaction:

During transaction execution, certain conditions may occur that necessitate cancellation of the transaction For example, data for the transaction may not be found

4. Concurrency control enforcement:

The concurrency control may decide to abort a transaction because it violates serializability or several transactions are in a state of deadlock

5. Disk failure:

Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash.

6. Physical problems and catastrophes:

Refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, overwriting disks or tapes by mistake

Failures of types 1, 2, 3, and 4 are more common than those of types 5 or 6.Whenever a failure of type 1 through 4 occurs, the system must keep sufficient information to quickly recover from the failure. Disk failure or other catastrophic failures of type 5 or 6 do not happen frequently; if they do occur, recovery is a major task.

5.2 Transaction and System Concepts

5.2.1 Transaction States and Additional Operations

A transaction is an atomic unit of work that should either be completed in its entirety or not done at all. For recovery purposes, the system keeps track of start of a transaction, termination, commit or aborts.

The main transaction control statements are:

BEGIN_TRANSACTION: marks the beginning of transaction execution

READ or WRITE: specify read or write operations on the database items that are executed as part of a transaction

END_TRANSACTION: specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution.

COMMIT_TRANSACTION: signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone

ROLLBACK: signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone, and the database is restored to its previous consistent state.

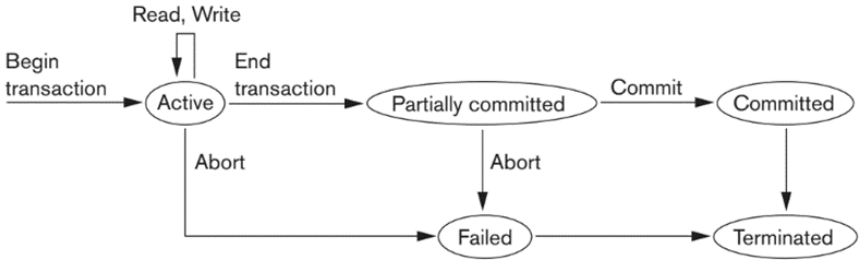


Figure 20.4
State transition diagram illustrating the states for transaction execution.

Transaction States (as shown in Figure 20.4)

- A transaction enters the Active state immediately after it starts execution.
  - In this state, it performs its **READ and WRITE** operations.
- When the transaction finishes executing its operations, it moves to the Partially Committed state.
  - At this stage, concurrency control protocols may perform additional checks.
  - Recovery protocols ensure that the transaction's updates can be safely recorded in the system log.
- If all required checks are successful, the transaction reaches the Commit point and enters the committed state.
  - In this state:
    - The transaction completes successfully.
    - All its changes are permanently stored in the database.
    - The changes must survive even if a system failure occurs.

Failure and Rollback

- A transaction may enter the Failed state if:
  - A concurrency control check fails, or
  - The transaction is aborted while in the Active state.
- When a transaction fails:
  - It must be rolled back.
  - All effects of its WRITE operations on the database are undone

Termination and Restart

- After completion (either committed or rolled back), the transaction enters the **Terminated state.**
  - The transaction leaves the system.

- Its information is removed from system tables.

· Failed or aborted transactions:

- Can be **restarted later** as new transactions.
- Restart may be automatic or initiated by the user

5.2.1.2 The System Log

· The log (or journal) keeps track of all transaction operations that affect the values of database items.

· This information is required to support recovery from transaction failures.

· The log is stored on disk, so it is not affected by system failures except in the case of disk failure or catastrophic failure.

· One or more main memory buffers hold the most recent portion of the log file.

· Log records are first written to the log buffer in main memory.

· When the log buffer becomes full, or when certain predefined conditions occur, the contents of the log buffer are appended to the end of the log file on disk. In addition, the log is periodically backed up to archival storage (such as tape) to protect against catastrophic failures.

Types of Log Records

In the following log records, **T** represents a unique transaction identifier generated by the system.

1. **[start_transaction, T]**
   Indicates that transaction **T** has started execution.

2. **[write_item, T, X, old_value, new_value]**
   Indicates that transaction **T** has changed the value of database item **X** from **old_value** to **new_value.**

3. **[read_item, T, X]**
   Indicates that transaction **T** has read the value of database item **X**.

4. **[commit, T]**
   Indicates that transaction **T** has completed successfully and its effects can be permanently recorded in the database.

5. **[abort, T]**
   Indicates that transaction **T** has been aborted and its effects must be undone.

5.2.1.3 Commit Point of a Transaction

Commit Point and Log Handling

- A transaction T reaches its commit point when:
  - All its database operations are executed successfully, and
  - All effects of its operations are recorded in the log.
- After reaching the commit point:
  - The transaction is said to be committed.
  - Its effects must be permanently stored in the database.
  - A **[commit, T]** record is written to the log.

Recovery Using Log

- After a system failure:
  - Transactions that have a **[start_transaction, T]** record but no **[commit, T]** record:
    - Must be rolled back (undone).
  - Transactions that have a **[commit, T]** record:
    - Must be redone using their log records.

Log Buffer and Disk Writing

- The log file is stored on disk.
- Log entries are first written into a log buffer in main memory.
- The log buffer is written to disk only:
  - When it is full, or
  - Under specific conditions.
- This reduces the overhead of repeated disk writes.

Force Writing Before Commit

- During a system crash:
  - Only log records already written to disk are used for recovery.
- Therefore, before a transaction is committed:
  - Any part of the log not yet written to disk must be written to disk.
- This process is called force-writing the log buffer to disk before commit.

5.2.1.4 DBMS-Specific Buffer Replacement Policies

Domain Separation (DS) Method

- Disk pages are divided into types such as:
  - Data pages
  - Index pages
  - Log pages
- The cache is divided into separate domains, each handling one type of page.
- Page replacement inside each domain uses LRU (Least Recently Used).
- Advantages:
  - Better performance than basic LRU.
- Limitation:
  - It is static and does not adapt to changing workloads.
- Variations:
  - GRU (Group LRU) replaces pages from the lowest-priority domain first.
  - Some methods dynamically adjust buffer allocation based on workload.

Hot Set Method

- Used for queries that repeatedly scan the same pages, such as nested-loop joins.

- Frequently accessed pages (hot set) are kept in memory and not replaced until processing finishes.
- This improves performance by avoiding repeated disk reads.

DBMIN Method

- Based on the Query Locality Set Model (QLSM).
- Predicts the pattern of page access for each query operation.
- Estimates the number of buffers needed for each file involved.
- Allocates buffers dynamically to each file based on its locality set.
- Similar to the working set concept in operating systems, but applied per file in a query

5.3 Desirable Properties of Transactions

Transactions must satisfy a set of important properties known as the **ACID properties**, which are enforced by the DBMS through its concurrency control and recovery mechanisms. These properties ensure that database transactions are executed reliably and maintain the correctness of the database.

The following are the ACID properties.

- **Atomicity**
  - A transaction is treated as a single atomic unit of processing.
  - It must either be executed completely or not executed at all.
  - Partial execution of a transaction is not allowed.
- **Consistency Preservation**
  - A transaction must preserve database consistency.
  - If it executes completely without interference from other transactions, it moves the database from one consistent state to another consistent state.
- **Isolation**

- A transaction should execute as if it is running in isolation from other transactions.
- Even when many transactions run concurrently, the execution of one transaction should not affect others.

- **Durability (Permanency)**
  - Once a transaction is committed, its changes must persist in the database.
  - These changes must not be lost even in the case of System failure, Power failure, Crash.

Characterizing Schedules based on Recoverability

Schedule: When transactions are executing concurrently in an interleaved fashion, then the order of execution of operations from all the various transactions is known as a schedule (or history).

- A schedule (or history) S of *n* transactions T_1, T_2, …, T_n is an ordering of the operations of these transactions.

- Operations from different transactions may be interleaved in the schedule. For each transaction T_i, the operations must appear in the same order as they occur in T_i itself.
- Total ordering: for any two operations in the schedule, one must occur before the other.
- Although schedules with partial ordering of operations are possible theoretically, total ordering is assumed for simplicity.

For recovery and concurrency control, the important transaction operations are:

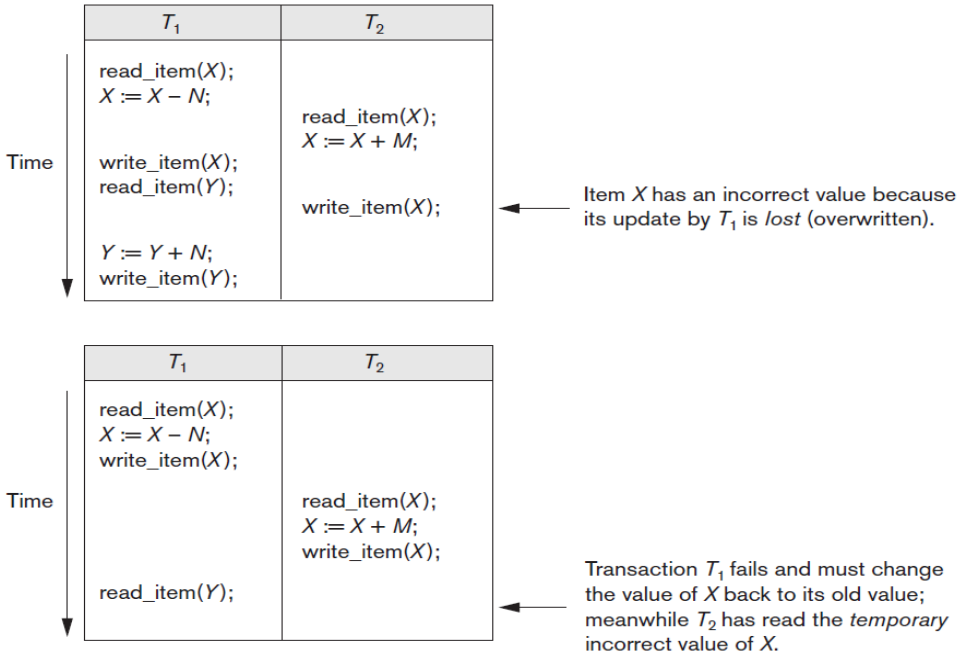
read_item(X)
write_item(X)
commit
abort

Notations for schedules:

- The transaction id is written as a subscript.
- The data item accessed appears in parentheses after r and w.
- Some schedules show only read/write operations, while others also include commit or abort.

Symbol	Operation
$\mathbf{b_i}$	begin_transaction of T_i
$\mathbf{r_i(X)}$	read_item(X) by T_i
$\mathbf{w_i(X)}$	write_item(X) by T_i
$\mathbf{e_i}$	end_transaction of T_i
$\mathbf{c_i}$	commit of T_i
$\mathbf{a_i}$	abort of T_i

Example Schedule S_a:



Sa: r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y); (first figure)

Transactions T_1 and T_2 are executed in an interleaved manner. Only read and write operations are shown.

Schedule Sb:

Sb: r1(X); w1(X); r2(X); w2(X); r1(Y); a1; (second figure)

Transaction T_1 aborts after performing read_item(Y).
The abort operation a_1 is explicitly shown.

Conflicting Operations in a Schedule:

Two operations in a schedule are said to conflict if they satisfy all three of the following conditions:

1. They belong to different transactions;
2. They access the same item X; and
3. At least one of the operations is a write_item(X).

Example:

Consider schedule Sa,

- The operations r1(X) and w2(X) conflict, as do the operations r2(X) and w1(X), and the operations w1(X) and w2(X).
- The operations r1(X) and r2(X) do not conflict, since they are both read operations;
- The operations w2(X) and w1(Y) do not conflict because they operate on distinct data items X and Y; and
- The operations r1(X) and w1(X) do not conflict because they belong to the same transaction.

Intuitively, two operations are conflicting if changing their order can result in a different outcome.

read-write conflict:

For example, if we change the order of the two operations
r1(X); w2(X) to
w2(X); r1(X),
then the value of X that is read by transaction T1 changes, because in the second ordering The value of X is read by r1(X) after it is changed by w2(X), whereas in the first ordering the value is read before it is changed. This is called a read-write conflict.

write-write conflict:

If we change the order of two operations such as
w1(X); w2(X) to
w2(X); w1(X)
For a write-write conflict, the last value of X will differ because in one case it is written by T2 and in the other case by T1.
Notice that two read operations are not conflicting because changing their order makes no difference in outcome.

Complete Schedule:

A schedule S of n transactions T1, T2, … , Tn is said to be a complete schedule if the following conditions hold:
1. The operations in S are exactly those operations in T1, T2, … , Tn, including a commit or abort operation as the last operation for each transaction in the schedule.
2. For any pair of operations from the same transaction Ti, their relative order of appearance in S is the same as their order of appearance in Ti.
3. For any two conflicting operations, one of the two must occur before the other in the schedule.

Partial Order in Transaction Schedules

- A partial order means that not all operations in a schedule need to be ordered.
- Non-conflicting operations can occur without specifying which one comes first.
- Therefore, the schedule does not define a fixed order for every pair of operations.

Characterizing Schedules Based on Recoverability:

- Recovery from failures depends on the type of schedule used. Some schedules allow easy and correct recovery, while others make recovery complex or even impossible.

- Therefore, it is important to classify schedules based on their recoverability. To preserve the durability property, a transaction that has been committed should never need to be rolled back.
- Schedules that guarantee this are called recoverable schedules. In a recoverable schedule, a transaction T is allowed to commit only after all transactions whose updates it has read have already committed.
- If a committed transaction may later need to be rolled back, the schedule is called nonrecoverable and should not be avoided by the DBMS.
- A transaction T is said to read from T′ if T reads a data item written by T′, provided no other committed write to that item occurs in between.
- The (partial) schedules Sa and Sb from the preceding section are both recoverable, since they satisfy the above definition.

Consider the schedule Sa′,
Sa′: r1(X); r2(X); w1(X); r1(Y); w2(X); c2; w1(Y); c1;
Sa′ is recoverable, even though it suffers from the lost update problem; this problem is handled by serializability theory.

Recoverable and Nonrecoverable Schedules: Example

Given Schedules

- **Sc:** r1(X); w1(X); r2(X); r1(Y); w2(X); c2; a1;
- **Sd:** r1(X); w1(X); r2(X); r1(Y); w2(X); w1(Y); c1; c2;
- **Se:** r1(X); w1(X); r2(X); r1(Y); w2(X); w1(Y); a1; a2;

Schedule Sc – Nonrecoverable

- Transaction T2 reads X written by T1.
- T2 commits before T1 commits.
- Later, T1 aborts.
- The value of X read by T2 becomes invalid.
- Since T2 is already committed, it would need to be rolled back.
- Rolling back a committed transaction is not allowed.
  Therefore, Sc is a nonrecoverable schedule.

Schedule Sd – Recoverable

- T2 reads X from T1, but
- T2 commits only after T1 commits.
- If T1 commits, the value read by T2 is valid.

- No committed transaction needs to be rolled back. Therefore, Sd is a recoverable schedule.

Schedule Se – Recoverable with Aborts

- T2 reads X from T1.

- T1 aborts instead of committing.
- T2 also aborts.
- Aborting T2 is safe because it has not been committed yet. Therefore, Se is recoverable.

Recoverable, Cascadeless, and Strict Schedules

Recoverable Schedules

> In a recoverable schedule, a committed transaction is never rolled back. This preserves the durability property of transactions. However, cascading rollback (cascading abort) can still occur. Cascading rollback happens when a transaction reads data from another transaction that later aborts, forcing it to abort as well.

Cascading Rollback (Cascading Abort)

Cascading Rollback Occurs when:

- Transaction T_2 reads an item written by T_1
- T_1 aborts
- T_2 must also be rolled back

This can involve many transactions, making recovery time-consuming.

Cascadeless Schedules

A schedule is cascadeless if a transaction reads only items written by committed transactions. Since all read values are from committed transactions, No cascading rollback occurs. To ensure this, read operations are delayed until the writing transaction commits or aborts.

Strict Schedules

A strict schedule is more restrictive than cascadeless. In a strict schedule, a transaction cannot read or write a data item X, until the last transaction that wrote X has committed or aborted.

Strict schedules simplify recovery. If a transaction aborts:

- The system can safely restore the before image (old value) of the data item.
- This simple undo method always works correctly for strict schedules.

Consider schedule Sf: w1(X, 5); w2(X, 8); a1;

Original value of X = 9
If T1 aborts, restoring the before image sets X back to 9
But T2 had already written X = 8
This leads to incorrect results
Hence, Sf is cascadeless but not strict

Characterizing Schedules Based on Serializability

Now we characterize the types of schedules that are always considered to be correct when concurrent transactions are executed. Such schedules are known as serializable schedules.

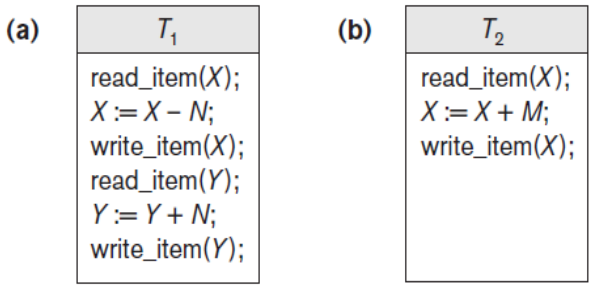


Figure 20.2
Two sample transactions.
(a) Transaction T_1.
(b) Transaction T_2.

Suppose that two users—for example, two airline reservations agents—submit to the DBMS transactions T1 and T2 in Figure 20.2 at approximately the same time. If no interleaving of operations is permitted, there are only two possible outcomes:

1. Execute all the operations of transaction T1 (in sequence) followed by all the operations of transaction T2 (in sequence).
2. Execute all the operations of transaction T2 (in sequence) followed by all the operations of transaction T1 (in sequence).

These two schedules are called serial schedules.
If interleaving of operations is allowed, there will be many possible orders in which the system can execute the individual operations of the transactions.
The concept of serializability of schedules is used to identify which schedules are correct when transaction executions have interleaving of their operations in the schedules.

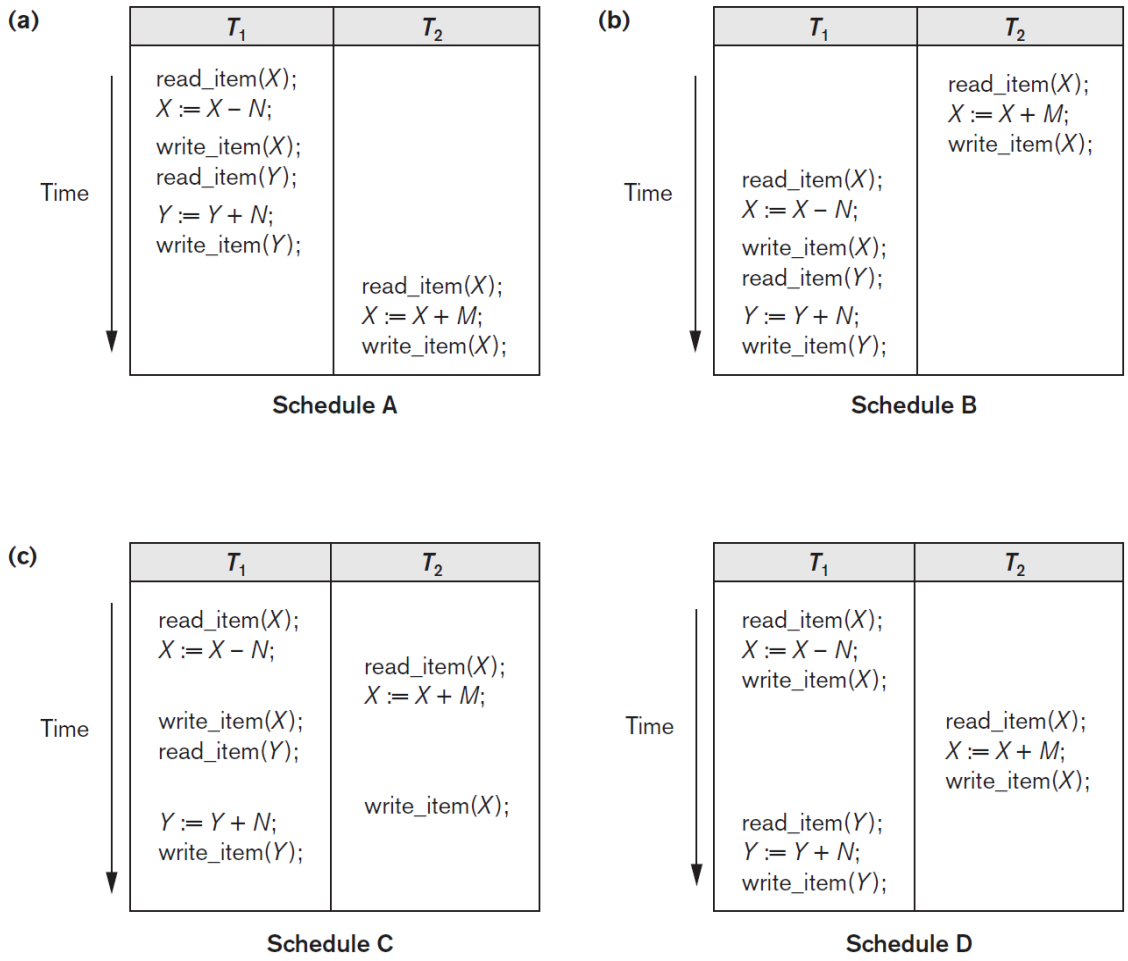


Figure 20.5
Examples of serial and nonserial schedules involving transactions T_1 and T_2. (a) Serial schedule A: T_1 followed by T_2. (b) Serial schedule B: T_2 followed by T_1. (c) Two nonserial schedules C and D with interleaving of operations.

Serial, Nonserial, and Conflict-Serializable Schedules

- Schedules A and B in Figures 20.5(a) and (b) are called serial because the operations of each transaction are executed consecutively, without any interleaved operations from the other transaction.
- In a serial schedule, entire transactions are performed in serial order: T1 and then T2 in Figure 20.5(a), and T2 and then T1 in Figure 20.5(b).
- Schedules C and D in Figure 20.5(c) are called nonserial because each sequence interleaves operations from the two transactions.

Serial Schedule: A schedule is serial if transactions execute one after another with no interleaving of operations (e.g., T1 fully executes before T2, or vice versa).

- In practice, DBMS allows interleaving of operations to improve performance and resource utilization. Serializability ensures correctness despite interleaving.
- A schedule is serializable if its effect on the database is equivalent to some serial schedule, even though operations may be interleaved.

- Serializable schedules are considered correct because they preserve database consistency as if transactions were executed serially.
- Serializability prevents problems such as:
    - ❖ Lost updates
    - ❖ Inconsistent reads
    - ❖ Incorrect final database states

Two Main Types of Serializability

- Conflict Serializability: Based on reordering non-conflicting operations
- View Serializability: Based on preserving read/write relationships (more general but harder to test)

Conflict Equivalence

Two schedules are conflict-equivalent if they contain the same operations and all conflicting operations occur in the same order.

Testing Conflict Serializability

Done using a precedence (serialization) graph:

- Nodes represent transactions
- Edges represent conflicts
- If the graph is **acyclic**, the schedule is conflict-serializable

Serializable Schedule: The definition of serializable schedule is as follows: A schedule S of n transactions is serializable if it is equivalent to some serial schedule of the same n transactions.

Correctness of Schedules

- A nonserial schedule is considered correct if and only if it is serializable.
- This is because serial schedules are inherently correct.
- To determine whether a nonserial schedule is serializable, we must define when two schedules are equivalent.

Result Equivalence

- Two schedules are result equivalent if they produce the same final database state.
- Limitations:
    - The same final state may occur by coincidence for specific initial values.
    - Does not guarantee equivalence for all database states.
    - Schedules may involve different transactions.
- Hence, result equivalence is unreliable for defining serializability.

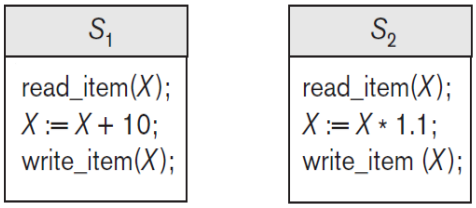


Figure 20.6
Two schedules that are result equivalent for the initial value of $X = 100$ but are not result equivalent in general.

Types of Schedule Equivalence:

1. Conflict Equivalence

- Two schedules are conflict equivalent if:
  - They contain the same operations of the same transactions.
  - The order of all conflicting operations is the same in both schedules.
- Conflicting operations:
  - Belong to different transactions
  - Operate on the same data item
  - At least one operation is a write
- Most commonly used definition in practice.

2. View Equivalence

- Two schedules are view equivalent if:
  - Every read operation reads the same value in both schedules.
  - The final write on each data item is done by the same transaction.
- More general than conflict equivalence.
- Harder to test and rarely used in implementations.

Conflict Equivalence of Two Schedules:

Two schedules are said to be conflict equivalent if the relative order of any two conflicting operations is the same in both schedules.

Conflicting Operations

Two operations conflict if all the following conditions hold:

- They belong to different transactions
- They access the same database item
- At least one of the operations is a write_item

Types of Conflicts

- Read–Write conflict (r–w)
- Write–Read conflict (w–r)

- Write–Write conflict (w–w)
- Read–Read operations do not conflict

If conflicting operations occur in different orders in two schedules, the effects may differ. Hence, such schedules are not conflict equivalent.

Examples:

Read–Write Conflict

Schedule S1: r1(X) → w2(X)

Schedule S2: w2(X) → r1(X)

Result:

r1(X) may read different values in S1 and S2

Not conflict equivalent

Write–Write Conflict

Schedule S1: w1(X) → w2(X)

Schedule S2: w2(X) → w1(X)

Result:

Final value of X may differ

Future reads of X may read different values

Not conflict equivalent.

Conflict Serializability:

A schedule S is said to be (conflict) serializable if it is conflict equivalent to some serial schedule S′. Conflict equivalence means that the relative order of all conflicting operations is the same in both schedules.

Reordering Operations

- In a nonserial schedule, nonconflicting operations can be reordered without changing the effect of the schedule.
- By repeatedly reordering only nonconflicting operations, a serial schedule S′ can be obtained.
- If such a serial schedule exists, the original schedule is serializable.

Refer to the figure 20.5. Schedule D is equivalent to serial schedule A (T1 → T2).

Observations:

- r2(X) reads the value of X written by T1 in both schedules.
- Other read_item operations read values from the initial database state.
- T1 is the last writer of Y in both schedules.

- T2 is the last writer of X in both schedules.

Since all conflicting operations occur in the same order, schedule D is conflict equivalent to A. Therefore, schedule D is serializable.
Operations r1(Y) and w1(Y) do not conflict with r2(X) and w2(X): They access different data items. Hence, they can be moved before r2(X) and w2(X) to form the serial order T1 → T2.

- Schedule C is not equivalent to either serial schedule:
    - T1 → T2
    - T2 → T1
- Conflicts preventing reordering:
    - r2(X) and w1(X) conflict

  Cannot move r2(X) down to form T1 → T2
    - w1(X) and w2(X) conflict

  Cannot move w1(X) down to form T2 → T1
- Since no equivalent serial schedule can be formed, schedule C is not serializable.

Testing for Serializability of a Schedule

- There is a simple algorithm for determining whether a particular schedule is (conflict) serializable or not.
- Algorithm 20.1 can be used to test a schedule for conflict serializability.
- The algorithm looks at only the read_item and write_item operations in a schedule to construct a precedence graph (or serialization graph), which is a directed graph G = (N, E) that consists of a set of nodes N = {T1, T2, … , Tn } and a set of directed edges E = {e1, e2, … , em }.
- There is one node in the graph for each transaction Ti in the schedule.
- Each edge ei in the graph is of the form (Tj → Tk), 1 ≤ j ≤ n, 1 ≤ k ≤ n, where Tj is the starting node of ei and Tk is the ending node of ei.
- Such an edge from node Tj to node Tk is created by the algorithm if a pair of conflicting operations exist in Tj and Tk and the conflicting operation in Tj appears in the schedule before the conflicting operation in Tk.

Algorithm 20.1. Testing Conflict Serializability of a Schedule S

1. For each transaction T_i participating in schedule S, create a node labeled T_i in the precedence graph.
2. For each case in S where T_j executes a read_item(X) after T_i executes a write_item(X), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
3. For each case in S where T_j executes a write_item(X) after T_i executes a read_item(X), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
4. For each case in S where T_j executes a write_item(X) after T_i executes a write_item(X), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles.

- The precedence graph is constructed as described in Algorithm 20.1.
- If there is a cycle in the precedence graph, schedule S is not (conflict) serializable; if there is no cycle, S is serializable.
- In the precedence graph, an edge from Ti to Tj means that transaction Ti must come before transaction Tj in any serial schedule that is equivalent to S, because two conflicting operations appear in the schedule in that order.
- If there is no cycle in the precedence graph, we can create an equivalent serial schedule S′ that is equivalent to S, by ordering the transactions that participate in S as follows:
    - Whenever an edge exists in the precedence graph from Ti to Tj, Ti must appear before Tj in the equivalent serial schedule S′.
    - Figure 20.7 shows such labels on the edges.

In general, several serial schedules can be equivalent to S if the precedence graph for S has no cycle. However, if the precedence graph has a cycle, it is easy to show that we cannot create any equivalent serial schedule, so S is not serializable.

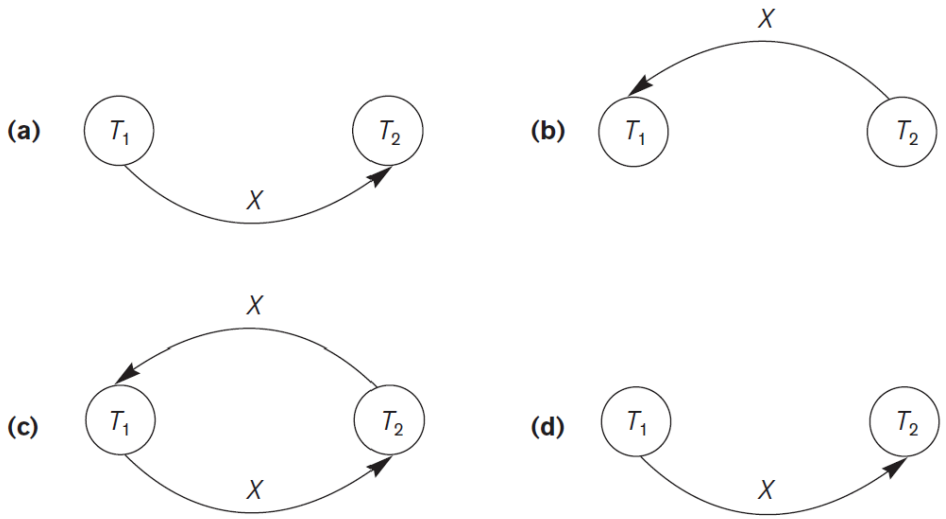


Figure 20.7
Constructing the precedence graphs for schedules A to D from Figure 20.5 to test for conflict serializability. (a) Precedence graph for serial schedule A. (b) Precedence graph for serial schedule B. (c) Precedence graph for schedule C (not serializable). (d) Precedence graph for schedule D (serializable, equivalent to schedule A).

Refer Figures 20.7(a) to (d).

- The graph for schedule C has a cycle, so it is not serializable.
- The graph for schedule D has no cycle, so it is serializable, and the equivalent serial schedule is T1 followed by T2.
- The graphs for schedules A and B have no cycles, as expected, because the schedules are serial and hence serializable

Another example, in which three transactions participate, is shown in Figure 20.8.

Figure 20.8
Another example of serializability testing. (a) The read and write operations of three transactions T_1, T_2, and T_3. (b) Schedule E. (c) Schedule F.

(a)

Transaction T_1	Transaction T_2	Transaction T_3
read_item(X);	read_item(Z);	read_item(Y);
write_item(X);	read_item(Y);	read_item(Z);
read_item(Y);	write_item(Y);	write_item(Y);
write_item(Y);	read_item(X);	write_item(Z);
	write_item(X);	

(b)

Time

Transaction T_1	Transaction T_2	Transaction T_3
	read_item(Z);	
	read_item(Y);	
	write_item(Y);	
		read_item(Y);
		read_item(Z);
read_item(X);		
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(X);	
read_item(Y);		
write_item(Y);		
	write_item(X);	

Schedule E

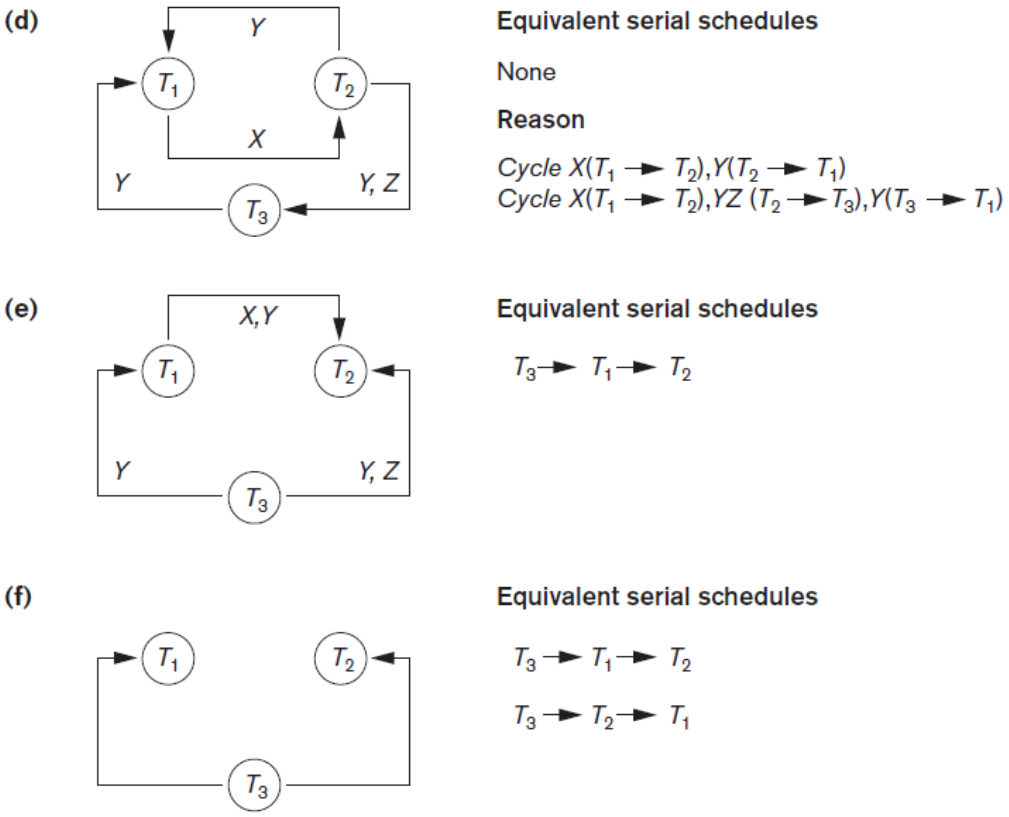
(c)

Time

Transaction T_1	Transaction T_2	Transaction T_3
		read_item(Y);
		read_item(Z);
read_item(X);		
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(Z);	
read_item(Y);		
write_item(Y);		
	read_item(Y);	
	write_item(Y);	
	read_item(X);	
	write_item(X);	

Schedule F

Figure 20.8 (continued)
Another example of serializability testing. (d) Precedence graph for schedule E. (e) Precedence graph for schedule F. (f) Precedence graph with two equivalent serial schedules.



Precedence Graphs for Schedules A–D (Figure 20.5)

- Schedules A and B:
  - Are already serial schedules
  - Their precedence graphs have no cycles. Therefore, they are serializable
- Schedule C:
  - Its precedence graph contains a cycle
  - Hence, schedule C is not serializable
- Schedule D:
  - Its precedence graph has no cycle
  - Therefore, schedule D is serializable
  - The equivalent serial schedule is T1 followed by T2

Three-Transaction Example (Figure 20.8)

- Figure 20.8(a) shows the **read_item** and **write_item** operations of three transactions.
- Two schedules are formed:
  - **Schedule E**
  - **Schedule F**

Analysis of Schedule E

- The precedence graph of schedule E contains cycles
- Hence, schedule E is not serializable
- No equivalent serial schedule exists

Analysis of Schedule F

- The precedence graph of schedule F has no cycles
- Hence, schedule F is serializable
- Only one equivalent serial schedule exists for F

Multiple Equivalent Serial Schedules

- In general:
    - A serializable schedule may have more than one equivalent serial schedule
- If the precedence graph allows multiple valid transaction orderings, then:
    - Multiple serial schedules are equivalent
- Figure 20.8(f) illustrates a precedence graph with two equivalent serial schedules

Finding an Equivalent Serial Schedule

Steps:

1. Select a transaction node with no incoming edges
2. Place it first in the serial order
3. Remove its outgoing edges from the graph
4. Repeat the process until all nodes are ordered
5. Ensure that no edge direction is violated

Figure 20.8(a) shows the read_item and write_item operations in each transaction. Two schedules E and F for these transactions are shown in Figures 20.8(b) and (c), respectively, and the precedence graphs for schedules E and F are shown in Figures 20.8(d) and (e). Schedule E is not serializable because the corresponding precedence graph has cycles. Schedule F is serializable, and the serial schedule equivalent to F is shown in Figure 20.8(e). Although only one equivalent serial schedule exists for F, in general there may be more than one equivalent serial schedule for a serializable schedule. Figure 20.8(f) shows a precedence graph representing a schedule that has two equivalent serial schedules. To find an equivalent serial schedule, start with a node that does not have any incoming edges, and then make sure that the node order for every edge is not violated.

How Serializability Is Used for Concurrency Control
Serial Schedules Are Inefficient because of the following reasons:

- Low CPU utilization (waiting for I/O).

- Long transactions delay others.
- Poor system throughput.

Advantages of serializable Schedules:

- Better resource utilization.
- Higher throughput.
- Maintains database consistency.

Testing serializability in practice is difficult because the interleaving of transaction operations is determined by the operating system scheduler.

- The order in which operations execute depends on factors such as system load, transaction submission time, and process priorities.
- Since these factors are dynamic and unpredictable, it is hard to determine in advance whether a schedule will be serializable.
- Moreover, checking serializability after execution is impractical because, if the schedule turns out to be nonserializable, the system may need to roll back transactions, which is costly and inefficient.
- To address this problem, DBMSs adopt concurrency control protocols instead of testing schedules after execution.
- These protocols enforce specific rules during transaction execution to ensure serializability automatically.
- This proactive approach avoids the need to analyze completed schedules and prevents incorrect executions from occurring in the first place.
- Since transactions are continuously entering the system, it is difficult to define the exact beginning and end of a schedule.
- Therefore, only the operations of committed transactions are considered. A schedule is regarded as serializable if its committed projection is equivalent to some serial schedule, because the DBMS guarantees correctness only for committed transactions.

Concurrency control protocols are used in practice:

- Two-Phase Locking (2PL)
- Snapshot Isolation
- Timestamp Ordering
- Multiversion Protocols
- Optimistic (Validation) Protocols

View Equivalence and View Serializability

Two schedules S and S′ are said to be view equivalent if the following three conditions hold:

1. The same set of transactions participates in S and S′, and S and S′ include the same operations of those transactions.
2. For any operation ri(X) of Ti in S, if the value of X read by the operation has been written by an operation wj(X) of Tj (or if it is the original value of X before the schedule started), the same condition must hold for the value of X read by operation ri(X) of Ti in S′.
3. If the operation wk(Y) of Tk is the last operation to write item Y in S, then wk(Y) of Tk must also be the last operation to write item Y in S′.

Other Types of Equivalence of Schedules

Serializability can be too restrictive for some applications where correctness does not require strict serial order.

In debit-credit transactions, updates involve addition and subtraction, which are commutative operations (order does not affect final result).

A schedule may be non-serializable but still correct if each read–update–write sequence is not improperly interrupted.

Traditional serializability ignores operation semantics, but correctness in some systems depends on the meaning of operations.

In long-duration or distributed applications (e.g., CAD systems), relaxed consistency models like eventual consistency are sometimes preferred over strict serializability.

Transaction Support in SQL

- With SQL, there is no explicit Begin_Transaction statement. Transaction initiation is done implicitly when particular SQL statements are encountered.
- Every transaction must have an explicit end statement, which is either a COMMIT or a ROLLBACK.
- Every transaction has certain characteristics attributed to it, specified by SET TRANSACTION statement in SQL.
- The characteristics are the access mode, the diagnostic area size, and the isolation level.
- The access mode of a transaction can be READ ONLY or READ WRITE. READ WRITE (default) allows select, insert, update, delete, and create operations. READ ONLY allows only data retrieval and does not permit any modifications.
- **Diagnostic Area Size (DIAGNOSTIC SIZE n)**
  This option specifies how many error or exception messages can be stored at a time. It keeps feedback information for the last *n* executed SQL statements.
- **Isolation Level**
  The isolation level controls how transactions are separated from each other. Possible levels

are READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE, with SERIALIZABLE as the default in most systems.

- If a transaction executes at a lower isolation level than SERIALIZABLE, then one or more of the following three violations may occur:
  - **Dirty read**: A transaction T1 may read the update of a transaction T2, which has not yet committed. If T2 fails and is aborted, then T1 would have read a value that does not exist and is incorrect.
  - **Nonrepeatable read**: A transaction T1 may read a given value from a table. If another transaction T2 later updates that value and T1 reads that value again, T1 will see a different value.
  - **Phantoms**: A transaction T1 may read a set of rows from a table, perhaps based on some condition specified in the SQL WHERE-clause. Now suppose that a transaction T2 inserts a new row r that also satisfies the WHERE-clause condition used in T1, into the table used by T1. The record r is called a phantom record because it was not there when T1 starts but is there when T1 ends. T1 may or may not see the phantom, a row that previously did not exist. If the equivalent serial order is T1 followed by T2, then the record r should not be seen; but if it is T2 followed by T1,then the phantom record should be in the result given to T1. If the system cannot ensure the correct behavior, then it does not deal with the phantom record problem.

Table 20.1 Possible Violations Based on Isolation Levels as Defined in SQL

	Type of Violation		
Isolation Level	**Dirty Read**	**Nonrepeatable Read**	**Phantom**
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

Table 20.1 summarizes the possible violations for the different isolation levels.

A sample SQL transaction might look like the following:

```
EXEC SQL WHENEVER SQLERROR GOTO UNDO;
EXEC SQL SET TRANSACTION
        READ WRITE
        DIAGNOSTIC SIZE 5
        ISOLATION LEVEL SERIALIZABLE;
EXEC SQL INSERT INTO EMPLOYEE (Fname, Lname, Ssn, Dno, Salary)
        VALUES ('Robert', 'Smith', '991004321', 2, 35000);
EXEC SQL UPDATE EMPLOYEE
        SET Salary = Salary * 1.1 WHERE Dno = 2;
```

```
EXEC SQL COMMIT;
GOTO THE_END;
UNDO: EXEC SQL ROLLBACK;
THE_END: ... ;
```

The above transaction consists of first inserting a new row in the EMPLOYEE table and then updating the salary of all employees who work in department 2. If an error occurs on any of the SQL statements, the entire transaction is rolled back. This implies that any updated salary (by this transaction) would be restored to its previous value and that the newly inserted row would be removed.

Concurrency Control in Databases:

- we discuss a number of concurrency control techniques that are used to ensure the noninterference or isolation property of concurrently executing transactions.
- One important set of protocols—known as two-phase locking protocols—employs the technique of locking data items to prevent multiple transactions from accessing the items concurrently.
- Locking protocols are used in some commercial DBMSs, but they are considered to have high overhead.
- Another set of concurrency control protocols uses timestamps. A timestamp is a unique identifier for each transaction, generated by the system. Timestamp values are generated in the same order as the transaction start times**.**
- Another extends timestamp order to two-phase locking,a protocol based on the concept of validation or certification of a transaction after it executes its operations.These are sometimes called optimistic protocols, and they also assume that multiple versions of a data item can exist.
- Snapshot isolation can utilize techniques similar to those proposed in validation-based and multiversion methods.
- the granularity of the data items—that is, what portion of the database a data item represents. An item can be as small as a single attribute (field) value or as large as a disk block, or even a whole file or the entire database**.**

Two-Phase Locking Techniques for Concurrency Control

- Some of the main techniques used to control concurrent execution of transactions are based on the concept of locking data items.
- A lock is a variable associated with a data item that describes the status of the item with respect to possible operations that can be applied to it.
- Generally, there is one lock for each data item in the database.
- Locks are used as a means of synchronizing the access by concurrent transactions to the database items.

21.1.1 Types of Locks and System Lock Tables

- ❖ Several types of locks are used in concurrency control. To introduce locking concepts gradually, first we discuss binary locks, which are simple but are also too restrictive for database concurrency control purposes and so are not used much.
- ❖ Then we discuss shared/exclusive locks—also known as read/write locks—which provide more general locking capabilities and are used in database locking schemes.

Binary Locks

- A binary lock can have two states or values: locked and unlocked (or 1 and 0, for simplicity). A distinct lock is associated with each database item X.

- If the value of the lock on X is 1, item X cannot be accessed by a database operation that requests the item. If the value of the lock on X is 0, the item can be accessed when requested, and the lock value is changed to 1. We refer to the current value (or state) of the lock associated with item X as lock(X).

Binary Lock Operations

- Two operations, lock_item and unlock_item, are used with binary locking.
- A transaction requests access to an item X by first issuing a lock_item(X) operation.
- If **LOCK(X) = 1, the transaction is forced to wait.**
- If **LOCK(X) = 0, it is set to 1 (the transaction locks the item) and the transaction is allowed to access item X.**
- When the transaction is through using the item, it issues an unlock_item(X) operation, which sets LOCK(X) back to 0 (unlocks the item) so that X may be accessed by other transactions. Hence, a binary lock enforces mutual exclusion on the data item.

```
lock_item(X):
B:  if LOCK(X) = 0              (*item is unlocked*)
        then LOCK(X) ←1         (*lock the item*)
    else
        begin
        wait (until LOCK(X) = 0
            and the lock manager wakes up the transaction);
        go to B
        end;
unlock_item(X):
    LOCK(X) ← 0;                (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
```

Figure 21.1
Lock and unlock operations for binary locks.

Lock Operation Properties

- The lock_item and unlock_item operations must be implemented as indivisible units (known as critical sections in operating systems).
- No interleaving should be allowed once a lock or unlock operation is started until the operation terminates or the transaction waits.
- The wait command within the lock_item(X) operation is usually implemented by putting the transaction in a waiting queue for item X until X is unlocked and the transaction can be granted access to it.
- Other transactions that also want to access X are placed in the same queue.

Lock Table

- It is simple to implement a binary lock.
- All that is needed is a binary-valued variable, LOCK, associated with each data item X in the database.
- In its simplest form, each lock can be a record with three fields: <Data_item_name, LOCK, Locking_transaction> plus a queue for transactions that are waiting to access the item.
- The system needs to maintain only these records for the items that are currently locked in a lock table.
- The lock table could be organized as a hash file on the item name.
- Items not in the lock table are considered to be unlocked.
- The DBMS has a lock manager subsystem to keep track of and control access to locks.

Rules for Binary Locking Scheme

If the simple binary locking scheme described here is used, every transaction must obey the following rules:

1. A transaction T must issue the operation lock_item(X) before any read_item(X) or write_item(X) operations are performed in T.

2. A transaction T must issue the operation unlock_item(X) after all read_item(X) and write_item(X) operations are completed in T.

3. A transaction T will not issue a lock_item(X) operation if it already holds the

lock on item X1.

4. A transaction T will not issue an unlock_item(X) operation unless it already

holds the lock on item X.

Shared/Exclusive (or Read/Write) Locks

· The preceding binary locking scheme is too restrictive for database items because at most one transaction can hold a lock on a given item.

· We should allow several transactions to access the same item X if they all access X for reading purposes only.

· This is because read operations on the same item by different transactions are not conflicting.

· However, if a transaction is to write an item X, it must have exclusive access to X.For this purpose, a different type of lock, called a multiple-mode lock, is used.

· In this scheme—called shared/exclusive or read/write locks—there are three locking operations: read_lock(X), write_lock(X), and unlock(X).

· A lock associated with an item X, LOCK(X), now has three possible states: read-locked, write-locked, or unlocked.

· A read-locked item is also called share-locked because other transactions are allowed to read the item.

· A write-locked item is called exclusive-locked because a single transaction exclusively holds the lock on the item.

One method for implementing the preceding operations on a read/write lock is to keep track of the number of transactions that hold a shared (read) lock on an item in the lock table, as well as a list of transaction ids that hold a shared lock.

· Each record in the lock table will have four fields: <Data_item_name, LOCK, No_of_reads, Locking_transaction(s)>.

· The system needs to maintain lock records only for locked items in the lock table.

· The value (state) of LOCK is either read-locked or write-locked, suitably coded.

· If LOCK(X) = write-locked, the value of locking_transaction(s) is a single transaction that holds the exclusive (write) lock on X.

· If LOCK(X) = read-locked, the value of locking_transaction(s) is a list of one or more transactions that hold the shared (read) lock on X.

```
read_lock(X):
B:  if LOCK(X) = "unlocked"
        then begin LOCK(X) ← "read-locked";
            no_of_reads(X) ← 1
            end
    else if LOCK(X) = "read-locked"
        then no_of_reads(X) ← no_of_reads(X) + 1
    else begin
            wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
            go to B
            end;
write_lock(X):
B:  if LOCK(X) = "unlocked"
        then LOCK(X) ← "write-locked"
    else begin
            wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
            go to B
            end;
unlock (X):
    if LOCK(X) = "write-locked"
        then begin LOCK(X) ← "unlocked";
                wakeup one of the waiting transactions, if any
                end
    else it LOCK(X) = "read-locked"
        then begin
                no_of_reads(X) ← no_of_reads(X) –1;
                if no_of_reads(X) = 0
                    then begin LOCK(X) = "unlocked";
                            wakeup one of the waiting transactions, if any
                            end
                end;
```

Figure 21.2
Locking and unlocking operations for two-mode (read/write, or shared/exclusive) locks.

When we use the shared/exclusive locking scheme, the system must enforce the following rules:

1. A transaction T must issue the operation read_lock(X) or write_lock(X) before any read_item(X) operation is performed in T.
2. A transaction T must issue the operation write_lock(X) before any write_item(X) operation is performed in T.
3. A transaction T must issue the operation unlock(X) after all read_item(X) and write_item(X) operations are completed in T.[3]

4. A transaction T will not issue a read_lock(X) operation if it already holds a read (shared) lock or a write (exclusive) lock on item X. This rule may be relaxed for downgrading of locks, as we discuss shortly.
5. A transaction T will not issue a write_lock(X) operation if it already holds a read (shared) lock or write (exclusive) lock on item X. This rule may also be relaxed for upgrading of locks, as we discuss shortly.
6. A transaction T will not issue an unlock(X) operation unless it already holds a read (shared) lock or a write (exclusive) lock on item X.

Conversion (Upgrading, Downgrading) of Locks

- It is desirable to relax conditions 4 and 5 in the preceding list in order to allow lock conversion; that is, a transaction that already holds a lock on item X is allowed under certain conditions to convert the lock from one locked state to another.
- For example, it is possible for a transaction T to issue a read_lock(X) and then later to upgrade the lock by issuing a write_lock(X) operation.
- If T is the only transaction holding a read lock on X at the time it issues the write_lock(X) operation, the lock can be upgraded; otherwise, the transaction must wait.
- It is also possible for a transaction T to issue a write_lock(X) and then later to downgrade the lock by issuing a read_lock(X) operation.
- When upgrading and downgrading of locks is used, the lock table must include transaction identifiers in the record structure for each lock (in the locking_transaction(s) field) to store the information on which transactions hold locks on the item.
- The descriptions of the read_lock(X) and write_lock(X) operations in Figure 21.2 must be changed appropriately to allow for lock upgrading and downgrading.
- Using binary locks or read/write locks in transactions, as described earlier, does not guarantee serializability of schedules on its own. Figure 21.3 shows an example where the preceding locking rules are followed but a nonserializable schedule may result. This is because in Figure 21.3(a) the items Y in T1 and X in T2 were unlocked too early. This allows a schedule such as the one shown in Figure 21.3(c) to occur, which is not a serializable schedule and hence gives incorrect results.
- To guarantee serializability, we must follow an additional protocol concerning the positioning of locking and unlocking operations in every transaction.

21.1.2 Guaranteeing Serializability by Two-Phase Locking

- A transaction is said to follow the two-phase locking protocol if all locking operations (read_lock, write_lock) precede the first unlock operation in the transaction.
- Such a transaction can be divided into two phases: an expanding or growing (first) phase, during which new locks on items can be acquired but none can be released; and a shrinking (second) phase, during which existing locks can be released but no new locks can be acquired.
- If lock conversion is allowed, then upgrading of locks (from read-locked to write-locked) must be done during the expanding phase, whereas downgrading of locks (from write-locked to read-locked) must be done in the shrinking phase.

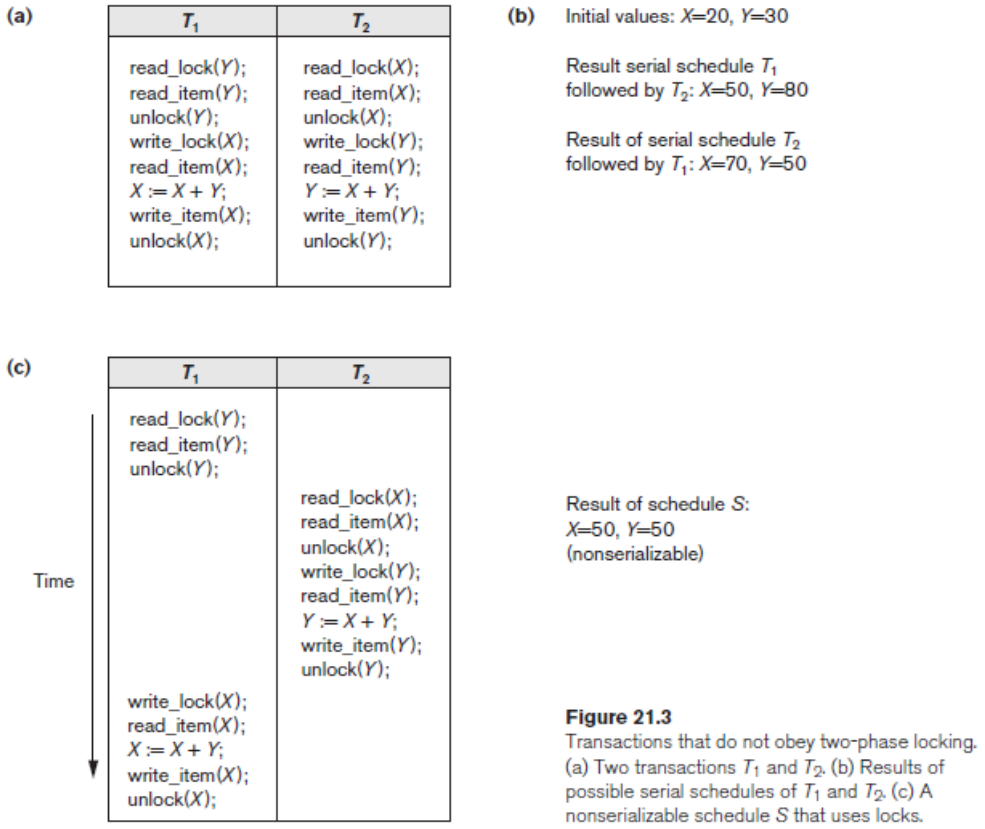


Figure 21.3
Transactions that do not obey two-phase locking. (a) Two transactions T_1 and T_2. (b) Results of possible serial schedules of T_1 and T_2. (c) A nonserializable schedule S that uses locks.

1. Transactions T1 and T2 in Figure 21.3(a) do not follow the two-phase locking protocol because the write_lock(X) operation follows the unlock(Y) operation in T1, and similarly the write_lock(Y) operation follows the unlock(X) operation in T2.

2. If we enforce two-phase locking, the transactions can be rewritten as T1′ and T2′, as shown in Figure 21.4.

3. Now, the schedule shown in Figure 21.3(c) is not permitted for T1′ and T2′ (with their modified order of locking and unlocking operations) under the rules of locking described in Section 21.1.1 because T1′ will issue its write_lock(X) before it unlocks item Y.

4. Consequently, when T2′ issues its read_lock(X), it is forced to wait until T1′ releases the lock by issuing an unlock(X) in the schedule.

5. However, this can lead to deadlock. It can be proved that, if every transaction in a schedule follows the two-phase locking protocol, the schedule is guaranteed to be serializable, obviating the need to test for serializability of schedules. The locking protocol, by enforcing two-phase locking rules, also enforces serializability.

6. Two-phase locking may limit the amount of concurrency that can occur in a schedule because a transaction T may not be able to release an item X after it is through using it if T

must lock an additional item Y later. Or, conversely, T must lock the additional item Y before it needs it so that it can release X.

7. Hence, X must remain locked by T until all items that the transaction needs to read or write have been locked.Only then can X be released by T.Meanwhile, another transaction seeking to access X may be forced to wait, even though T is done with X.Conversely, if Y is locked earlier than it is needed, another transaction seeking to access Y is forced to wait even though T is not using Y yet.

8. This is the price for guaranteeing serializability of all schedules without having to check the schedules themselves.

9. Although the two-phase locking protocol guarantees serializability (that is, every schedule that is permitted is serializable), it does not permit all possible serializable schedules (that is, some serializable schedules will be prohibited by the protocol).

Basic, Conservative, Strict, and Rigorous Two-Phase Locking

Basic 2PL

- There are a number of variations of two-phase locking (2PL).
- The technique just described is known as **basic 2PL.**
- A variation known as conservative 2PL (or static 2PL) requires a transaction to lock all the items it accesses before the transaction begins execution, by predeclaring its read-set and write-set.
- Recall from Section 21.1.2 that the read-set of a transaction is the set of all items that the transaction reads, and the write-set is the set of all items that it writes.
- If any of the predeclared items needed cannot be locked, the transaction does not lock any item; instead, it waits until all the items are available for locking.
- Conservative 2PL is a deadlock-free protocol, as we will see in Section 21.1.3 when we discuss the deadlock problem.
- However, it is difficult to use in practice because of the need to predeclare the read-set and write-set, which is not possible in some situations.

Strict 2PL

- ➢ In practice, the most popular variation of 2PL is strict 2PL, which guarantees strict schedules.
- ➢ In this variation, a transaction T does not release any of its write-locks until after it commits or aborts.
- ➢ Hence, no other transaction can read or write an item that is written by T unless T has committed, leading to a strict schedule for recoverability.

Strict 2PL is not deadlock-free.

A more restrictive variation of strict 2PL is rigorous 2PL, which also guarantees strict schedules.

In this variation, a transaction T does not release any of its locks (exclusive or shared) until after it commits or aborts, and so it is easier to implement than strict 2PL.

difference between strict and rigorous 2PL:

1. The former holds write-locks until it commits, whereas the latter holds all locks (read and write).

2. Also, the difference between conservative and rigorous 2PL is that the former must lock all its items before it starts, so once the transaction starts it is in its shrinking phase.

3. The latter does not unlock any of its items until after it terminates (by committing or aborting), so the transaction is in its expanding phase until it ends

Usually, the concurrency control subsystem itself is responsible for generating the read_lock and write_lock requests.For example, suppose the system is to enforce the strict 2PL protocol.Then, whenever transaction T issues a read_item(X), the system calls the read_lock(X) operation on behalf of T.If the state of LOCK(X) is write_locked by some other transaction T′, the system places T in the waiting queue for item X.Otherwise, it grants the read_lock(X) request and permits the read_item(X) operation of T to execute.

- On the other hand, if transaction T issues a write_item(X), the system calls the write_lock(X) operation on behalf of T.
- If the state of LOCK(X) is write_locked or read_locked by some other transaction T′, the system places T in the waiting queue for item X.
- If the state of LOCK(X) is read_locked and T itself is the only transaction holding the read lock on X, the system upgrades the lock to write_locked and permits the write_item(X) operation by T.
- Finally, if the state of LOCK(X) is unlocked, the system grants the write_lock(X) request and permits the write_item(X) operation to execute.
- After each action, the system must update its lock table appropriately.
- The use of locks can also cause two additional problems: deadlock and starvation.

21.1.3 Dealing with Deadlock and Starvation

- Deadlock occurs when each transaction T in a set of two or more transactions is waiting for some item that is locked by some other transaction T′ in the set.
- Hence, each transaction in the set is in a waiting queue, waiting for one of the other transactions in the set to release the lock on an item.
- But because the other transaction is also waiting, it will never release the lock.
- A simple example is shown in Figure 21.5(a), where the two transactions T1′ and T2′ are deadlocked in a partial schedule.
- T1′ is in the waiting queue for X, which is locked by T2′, whereas T2′ is in the waiting queue for Y, which is locked by T1′.

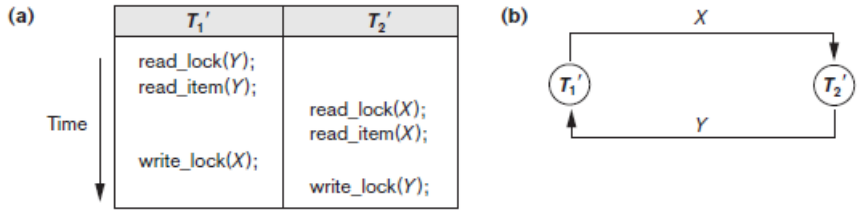


Figure 21.5
Illustrating the deadlock problem. (a) A partial schedule of T_1' and T_2' that is in a state of deadlock. (b) A wait-for graph for the partial schedule in (a).

Deadlock Prevention Protocols

- One way to prevent deadlock is to use a deadlock prevention protocol.
- One deadlock prevention protocol, which is used in conservative two-phase locking, requires that every transaction lock all the items it needs in advance.If any of the items cannot be obtained, none of the items are locked.
- Rather, the transaction waits and then tries again to lock all the items it needs. This solution further limits concurrency.
- A second protocol, which also limits concurrency, involves ordering all the items in the database and making sure that a transaction that needs several items will lock them according to that order.
- This requires that the programmer (or the system) is aware of the chosen order of the items, which is also not practical in the database context.
- Many other deadlock prevention schemes have been proposed that make a decision about what to do with a transaction involved in a possible deadlock situation.
- Some of these techniques use the concept of transaction timestamp TS(T′), which is a unique identifier assigned to each transaction.

- The timestamps are typically based on the order in which transactions are started. Hence, if transaction T1 starts before transaction T2, then TS(T1) < TS(T2). The older transaction (which starts first) has the smaller timestamp value.
- Two schemes that prevent deadlock are called **wait-die and wind-wait**.
- Suppose that transaction Ti tries to lock an item X but is not able to because X is locked by some other transaction Tj with a conflicting lock.

- **Wait-die.** If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) T_i is allowed to wait; otherwise (T_i younger than T_j) abort T_i (T_i *dies*) and restart it later *with the same timestamp.*
- **Wound-wait.** If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) abort T_j (T_i *wounds* T_j) and restart it later *with the same timestamp;* otherwise (T_i younger than T_j) T_i is allowed to wait.

Wait–Die and Wound–Wait Schemes

1. Wait–Die Scheme

· An older transaction can wait for a younger transaction.

· A younger transaction requesting a lock held by an older transaction is aborted and restarted.

2. Wound–Wait Scheme

· A younger transaction can wait for an older transaction.

· An older transaction requesting a lock held by a younger transaction aborts the younger transaction.

· In both schemes, the younger transaction is aborted.

· These techniques are deadlock-free.

· In wait–die, transactions only wait for younger transactions, so no cycle is created.

· In wound–wait, transactions only wait for older transactions, so no cycle is created.

· Sometimes transactions may be aborted and restarted unnecessarily.

3. No Waiting (NW) Algorithm

· If a transaction cannot get a lock, it is immediately aborted.

· The transaction is restarted later after a delay.

· Since no transaction waits, deadlock cannot occur.

· But this may cause many unnecessary aborts and restarts.

4. Cautious Waiting (CW) Algorithm

If transaction Ti requests a lock on item X but X is locked by Tj:

Rule

· If Tj is not waiting for another item, then Ti waits.

- If Tj is already waiting, then Ti is aborted.
- A transaction never waits for another blocked transaction.
- Because of this, deadlock cannot occur.

■ **Cautious waiting.** If T_j is not blocked (not waiting for some other locked item), then T_i is blocked and allowed to wait; otherwise abort T_i.

Deadlock Detection, Timeout, and Starvation

- When a transaction releases the lock on the item that another transaction was waiting for, the directed edge is dropped from the wait-for graph. We have a state of deadlock if and only if the wait-for graph has a cycle.One problem with this approach is determining when the system should check for a deadlock.
- One possibility is to check for a cycle every time an edge is added to the wait-for graph, but this may cause excessive overhead.
- Criteria such as the number of currently executing transactions or the period of time several transactions have been waiting to lock items may be used instead to check for a cycle.
- If the system is in a state of deadlock, some of the transactions causing the deadlock must be aborted.
- Choosing which transactions to abort is known as victim selection.
- The algorithm for victim selection should generally avoid selecting transactions that have been running for a long time and that have performed many updates.
- Instead, it should try to select transactions that have not made many changes (younger transactions).

- **Timeouts**
- Another simple scheme to deal with deadlock is the use of timeouts.
- This method is practical because of its low overhead and simplicity.
- In this method, if a transaction waits longer than a system-defined timeout period, the system assumes that the transaction may be deadlocked and aborts it.
- This happens regardless of whether a deadlock actually exists.

- **Starvation**
- Another problem that may occur when locking is used is starvation.Starvation occurs when a transaction cannot proceed for an indefinite period of time while other transactions in the system continue normally.
- This may occur if the waiting scheme for locked items is unfair and gives priority to some transactions over others.
- One solution for starvation is to use a fair waiting scheme, such as a first-come-first-served queue.
- In this scheme, transactions are allowed to lock an item in the order in which they originally requested the lock.

- Another scheme allows some transactions to have priority over others but increases the priority of a transaction the longer it waits, until it eventually gets the highest priority and proceeds.
- Starvation can also occur because of victim selection if the algorithm repeatedly selects the same transaction as the victim, causing it to abort and never finish execution.
- The algorithm can use higher priorities for transactions that have been aborted multiple times to avoid this problem.
- The wait-die and wound-wait schemes avoid starvation because they restart an aborted transaction with its original timestamp, so the possibility that the same transaction is aborted repeatedly is small.

21.2 Concurrency Control Based on Timestamp

- The use of locking, combined with the two-phase locking protocol, guarantees serializability of schedules. The serializable schedules produced by two-phase locking have their equivalent serial schedules based on the order in which executing transactions lock the items they acquire.
- If a transaction needs an item that is already locked, it may be forced to wait until the item is released.
- Some transactions may be aborted and restarted because of the deadlock problem.A different approach to concurrency control involves using transaction timestamps to order transaction execution for an equivalent serial schedule.

21.2.1 Timestamps

- Recall that a timestamp is a unique identifier created by the DBMS to identify a transaction.
- Typically, timestamp values are assigned in the order in which the transactions are submitted to the system. Therefore, a timestamp can be thought of as the transaction start time.
- Refer the timestamp of transaction T as TS(T). Concurrency control techniques based on timestamp ordering do not use locks.Hence, deadlocks cannot occur.

Generating Timestamps

- Timestamps can be generated in several ways.One possibility is to use a counter that is incremented each time its value is assigned to a transaction.The transaction timestamps are numbered 1, 2, 3, and so on in this scheme.

- A computer counter has a finite maximum value.Therefore, the system must periodically reset the counter to zero when no transactions are executing for some short period of time.
- Another way to implement timestamps is to use the current date and time value of the system clock. In this method, the system ensures that no two timestamp values are generated during the same clock tick.

21.2.2 The Timestamp Ordering Algorithm for Concurrency Control

· The idea for this scheme is to enforce the equivalent serial order on the transactions based on their timestamps.A schedule in which the transactions participate is then serializable.

The only equivalent serial schedule permitted has the transactions in order of their timestamp values.This is called timestamp ordering (TO).

This differs from two-phase locking. In two-phase locking, a schedule is serializable by being equivalent to some serial schedule allowed by the locking protocols.

· In timestamp ordering, the schedule is equivalent to the particular serial order corresponding to the order of the transaction timestamps.

· The algorithm allows interleaving of transaction operations.However, it must ensure that for each pair of conflicting operations in the schedule, the order in which the item is accessed must follow the timestamp order.

1. **read_TS(*X*).** The **read timestamp** of item *X* is the largest timestamp among all the timestamps of transactions that have successfully read item *X*—that is, read_TS(*X*) = TS(*T*), where *T* is the *youngest* transaction that has read *X* successfully.
2. **write_TS(*X*).** The **write timestamp** of item *X* is the largest of all the time-stamps of transactions that have successfully written item *X*—that is, write_TS(*X*) = TS(*T*), where *T* is the *youngest* transaction that has written *X* successfully. Based on the algorithm, *T* will also be the last transaction to write item *X*, as we shall see.

Basic Timestamp Ordering (TO)

- Whenever a transaction T issues read_item(X) or write_item(X), the timestamp of T is compared with read_TS(X) and write_TS(X).This check ensures that the timestamp order of transactions is not violated.

- If the timestamp order is violated, the transaction T is aborted and restarted with a new timestamp. If T is rolled back, any transaction T1 that used a value written by T must also be rolled back. Similarly, any transaction T2 that used a value written by T1 must also be rolled back.This process is called cascading rollback.
- Cascading rollback is a problem in basic Timestamp Ordering, because the schedules produced are not guaranteed to be recoverable.Therefore, an additional protocol is required to ensure schedules are recoverable, cascadeless, or strict.

The concurrency control algorithm must check whether conflicting operations violate timestamp ordering in two cases:

· read_item(X) operation

· write_item(X) operation

1. Whenever a transaction T issues a write_item(X) operation, the following check is performed:
   a. If read_TS(X) > TS(T) or if write_TS(X) > TS(T), then abort and roll back T and reject the operation. This should be done because some *younger* transaction with a timestamp greater than TS(T)—and hence *after* T in the timestamp ordering—has already read or written the value of item X before T had a chance to write X, thus violating the timestamp ordering.
   b. If the condition in part (a) does not occur, then execute the write_item(X) operation of T and set write_TS(X) to TS(T).
2. Whenever a transaction T issues a read_item(X) operation, the following check is performed:
   a. If write_TS(X) > TS(T), then abort and roll back T and reject the operation. This should be done because some younger transaction with timestamp greater than TS(T)—and hence *after* T in the timestamp ordering—has already written the value of item X before T had a chance to read X.
   b. If write_TS(X) ≤ TS(T), then execute the read_item(X) operation of T and set read_TS(X) to the *larger* of TS(T) and the current read_TS(X).

Whenever the basic TO algorithm detects two *conflicting operations* that occur in the incorrect order, it rejects the later of the two operations by aborting the transaction that issued it. The schedules produced by basic TO are hence guaranteed to be *conflict serializable*. As mentioned earlier, deadlock does not occur with timestamp ordering.

Strict Timestamp Ordering (TO)

- Strict Timestamp Ordering is a variation of the basic Timestamp Ordering (TO) method. It ensures that schedules are strict and conflict serializable.This helps in easy recoverability of transactions.
- In this method, when a transaction T issues read_item(X) or write_item(X) and TS(T) > write_TS(X), the operation cannot execute immediately.he read or write operation of T is

delayed.The operation waits until the transaction T′ that wrote X (where TS(T′) = write_TS(X)) commits or aborts.

Thomas's Write Rule. A modification of the basic TO algorithm, known as **Thomas's write rule**, does not enforce conflict serializability, but it rejects fewer write operations by modifying the checks for the write_item(X) operation as follows:

1. If read_TS(X) > TS(T), then abort and roll back T and reject the operation.
2. If write_TS(X) > TS(T), then do not execute the write operation but continue processing. This is because some transaction with timestamp greater than TS(T)—and hence after T in the timestamp ordering—has already written the value of X. Thus, we must ignore the write_item(X) operation of T because it is already outdated and obsolete. Notice that any conflict arising from this situation would be detected by case (1).
3. If neither the condition in part (1) nor the condition in part (2) occurs, then execute the write_item(X) operation of T and set write_TS(X) to TS(T).

Database Recovery Techniques:

Recovery Concepts

22.1.1 Recovery Outline and Categorization of Recovery Algorithms

1. Meaning of Recovery

1. Recovery is the process of restoring the database to the most recent consistent state before failure.
2. Failures may occur due to transaction errors, system crashes, or disk failures.
3. To perform recovery, the DBMS maintains information about all changes made by transactions.
4. This information is stored in a system log (transaction log).
5. The log file records every update operation performed by transactions.

2. Recovery Strategies

Recovery methods depend on the type of failure.

1. Recovery from Catastrophic Failures

1. Catastrophic failure means severe damage to the database (e.g., disk crash).
2. In this situation, the database stored on disk becomes physically damaged.
3. Recovery is done using a backup copy of the database stored in archival storage.
4. Archival storage may include:
    - Magnetic tapes
    - External storage systems
    - Large capacity offline storage
5. The recovery steps include:
    - Restore the previous backup copy of the database.
    - Use the log file to redo the operations of committed transactions.
6. This reconstruction continues up to the time of failure.

2. Recovery from Non-Catastrophic Failures

1. In this case, the database on disk is not physically damaged.
2. Failures may include:
    - Transaction failure
    - System crash
    - Power failure
    - Software errors
3. The recovery system analyzes the log file stored on disk.
4. The goal is to detect operations that caused inconsistency.
5. Two important actions are used:
    - UNDO
    - REDO

UNDO

1. Used when a transaction has not committed.
2. If such a transaction modified database items, its changes must be reversed.
3. This ensures the database returns to the previous correct state.

REDO

1. Used when a transaction has committed, but its updates were not yet written to disk.
2. The recovery process reapplies those operations from the log.
3. This ensures all committed changes are reflected in the database.
4. For non-catastrophic failures, a full backup is not required.
5. Recovery is performed using online system logs.

3. Main Recovery Policies

There are two major recovery policies used in DBMS:

1. Deferred Update
2. Immediate Update

4. Deferred Update Technique (NO-UNDO / REDO)

Concept

1. In deferred update, the database is not updated immediately on disk.
2. Updates are applied to the database only after the transaction commits.

Process

1. All updates are stored in:
    - Local transaction workspace, or
    - Main memory buffers (DBMS cache).
2. Before commit, update operations are recorded in the log file on disk.
3. After the commit point, updates are written from memory buffers to the database.

Failure Handling

1. If a transaction fails before commit, it has not changed the database on disk.
2. Therefore:
    - UNDO is not required.
3. However, if a committed transaction's updates are not yet written to disk, they must be REDONE from the log.

Algorithm Name

- Deferred update is called the NO-UNDO / REDO algorithm.

5. Immediate Update Technique (UNDO / REDO)

1. In immediate update, database items may be updated before the transaction commits.
2. Updates may be written directly to the database.

Important Rule

1. Before writing changes to the database, the update must be recorded in the log file.
2. The log record must be force-written to disk first.

Failure Handling

1. If a transaction fails before commit, some changes may already exist in the database.
2. These changes must be undone.
3. If a committed transaction's updates are missing on disk, they must be redone.

Algorithm Name

- This technique is called the UNDO / REDO algorithm.
- It is commonly used in practical database systems.

6. Variation: UNDO / NO-REDO Algorithm

1. Another variation requires that all updates must be written to the database before commit.
2. Because updates are already stored on disk before commit:
    - REDO is not required.
3. Only UNDO is required if a transaction fails.
4. This technique is called the UNDO / NO-REDO algorithm.

7. Idempotent Property of Recovery Operations

Idempotent operation means performing an operation multiple times gives the same result as performing it once.

Importance in Recovery

1. UNDO and REDO operations must be idempotent.
2. During recovery, the system might crash again while recovery is running.
3. When recovery restarts, the same operations might be executed again.

Caching (Buffering) of Disk Blocks

1. Concept of Caching

- Caching (buffering) means temporarily storing database disk pages in main memory buffers before writing them back to disk.
- It improves database performance and recovery efficiency.

- Though buffering is normally handled by the operating system, in DBMS it is managed by the DBMS using low-level OS routines.

2. DBMS Cache

- DBMS maintains a collection of main memory buffers called the DBMS cache.
- These buffers store disk pages containing database items.
- A cache directory keeps track of the pages in the cache.
- Format
  <Disk_page_address, Buffer_location, ...>

3. Accessing Data Using Cache

1. When the DBMS requests a data item, it first checks the cache directory.
2. If the required page is present in cache, it is accessed directly.
3. If not present, the page is read from disk and copied into the cache.
4. If the cache is full, some buffers are replaced (flushed) to make space.

4. Dirty Bit

- Each buffer has a dirty bit indicating whether the page has been modified.
- Dirty bit = 0: Page not modified.
- Dirty bit = 1: Page modified in memory.
- When a modified page is replaced, it must be written back to disk.

5. Pin–Unpin Bit

- A pin bit indicates whether a page can be written back to disk.
- Pinned (1): Page cannot be written to disk yet (transaction still using it).
- Unpinned (0): Page can be replaced or written to disk.

6. Buffer Flushing Strategies

1. In-Place Updating

- Modified buffer is written to the same disk location, overwriting the old data.
- Only one copy of each disk block is maintained.

2. Shadowing

- Updated buffer is written to a different disk location.
- Multiple versions of data blocks are maintained.
- Rarely used in practice.

7. Before Image and After Image

- Before Image (BFIM): Old value of a data item before update.

- After Image (AFIM): New value of the data item after update.
- If shadowing is used, both BFIM and AFIM can be stored on disk, reducing the need for logging.

22.1.3 Write-Ahead Logging (WAL), Steal/No-Steal, Force/No-Force

1. Write-Ahead Logging (WAL)

- When in-place updating is used, a log file is required for recovery.
- The Before Image (BFIM) of a data item must be recorded in the log before it is overwritten by the After Image (AFIM) on disk.
- The log entry must be force-written to disk before the database page is updated.
- This rule is called Write-Ahead Logging (WAL) and ensures UNDO is possible during recovery.

Types of Log Entries

- REDO log entry
    - Contains the AFIM (new value) of the data item.
    - Used to redo the effect of an operation.
- UNDO log entry
    - Contains the BFIM (old value) of the data item.
    - Used to undo the effect of an operation.
- UNDO/REDO log entry
    - Contains both BFIM and AFIM in a single record.
- If cascading rollback is possible, read_item operations are also recorded as UNDO-type entries.

2. Log Storage in DBMS Cache

- The DBMS cache stores:
    - Data file blocks
    - Index blocks
    - Log blocks
- Log records are written to a log buffer in main memory.
- The log is a sequential append-only disk file.
- Several recent log blocks may remain in memory buffers.

WAL Rule

- The log record must be written to disk before the corresponding data block is written to disk.

3. Steal / No-Steal Policy

No-Steal

- Updated buffer cannot be written to disk before the transaction commits.
- The page remains pinned in the cache.
- UNDO is not required during recovery because uncommitted updates never reach disk.

Steal

- Updated buffer may be written to disk before the transaction commits.
- Used when the buffer manager needs space for another page.
- Requires UNDO during recovery.

4. Force / No-Force Policy

Force

- All updated pages of a transaction are written to disk before commit.
- REDO is not required during recovery.

No-Force

- Updated pages may remain in memory after commit.
- Requires REDO during recovery.

5. Typical Recovery Strategy

- Most DBMS systems use Steal + No-Force strategy.
- Advantages:
    - Steal: reduces large buffer requirements.
    - No-Force: reduces repeated disk I/O operations when pages are frequently updated.

6. WAL Protocol for UNDO/REDO Recovery

1. The BFIM must be written to the log and flushed to disk before the database page is overwritten.
2. A transaction cannot commit until all its UNDO and REDO log records are written to disk.

7. Transaction Lists Maintained by DBMS

The recovery subsystem may maintain:

- Active transaction list – transactions that started but not committed.
- Committed transaction list.
- Aborted transaction list since the last checkpoint.

These lists help improve recovery efficiency.

22.1.4 Checkpoints in System Log and Fuzzy Check pointing

1. Checkpoint

- A checkpoint is a special log record written periodically.
- Format: [checkpoint, list of active transactions].

Purpose

- At checkpoint time, the system writes all modified buffers to disk.
- Therefore, transactions committed before the checkpoint do not need REDO after a crash.

Information Stored

- List of active transaction IDs.
- Sometimes includes addresses of first and last log records for each active transaction.

2. Checkpoint Interval

The recovery manager decides checkpoint intervals based on:

- Time interval (e.g., every m minutes), or
- Number of committed transactions since last checkpoint.

3. Checkpoint Process

1. Temporarily suspend transaction execution.
2. Write all modified memory buffers to disk.
3. Write [checkpoint] record to log and flush log to disk.
4. Resume transaction processing.

4. Fuzzy Checkpointing

- Traditional checkpointing delays transactions while buffers are written to disk.
- Fuzzy checkpointing avoids this delay.

Process

1. Write [begin_checkpoint] record to the log.
2. Resume transaction processing immediately.
3. Write modified buffers to disk in the background.
4. Write [end_checkpoint] record after completion.

Additional Mechanism

- A pointer file on disk indicates the current valid checkpoint.
- The pointer is updated after checkpoint completion.

22.1.5 Transaction Rollback and Cascading Rollback

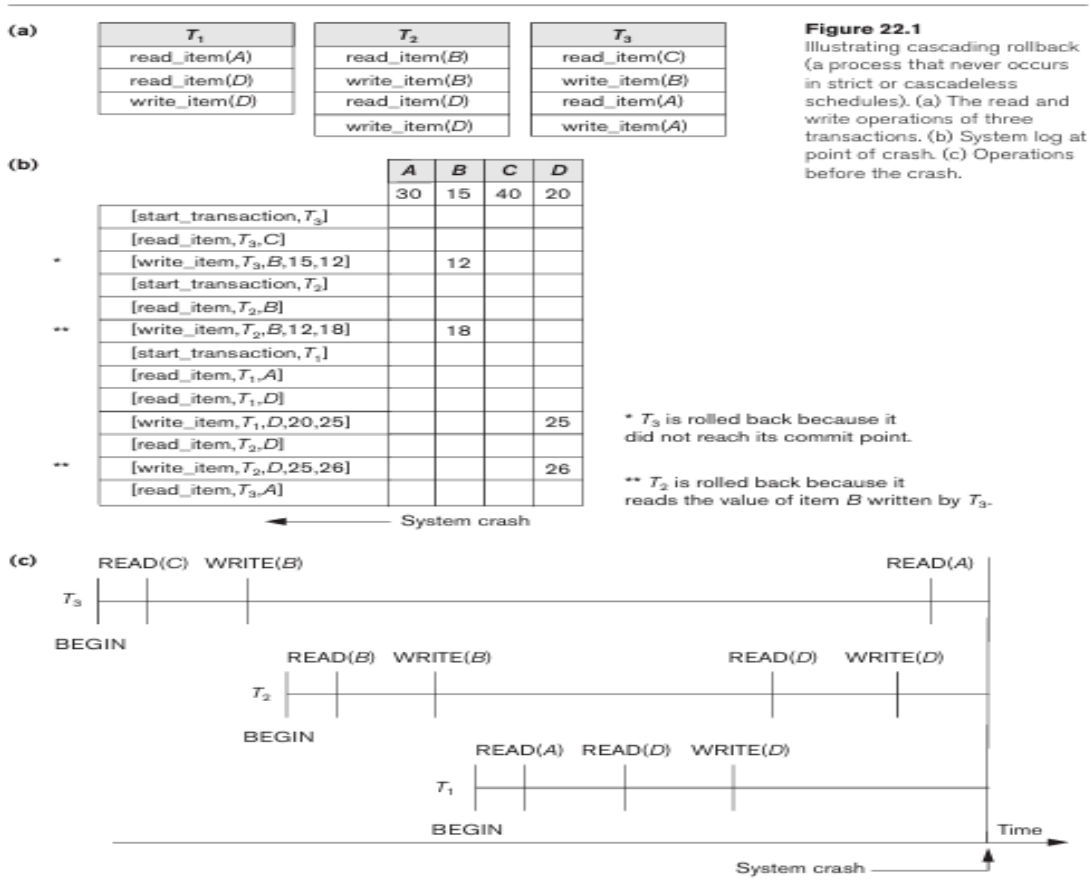


Figure 22.1
Illustrating cascading rollback (a process that never occurs in strict or cascadeless schedules). (a) The read and write operations of three transactions. (b) System log at point of crash. (c) Operations before the crash.

1. Transaction Rollback

- If a transaction fails before commit, its changes must be reversed.
- Updated data items on disk are restored using BFIM from UNDO log entries.

2. Cascading Rollback

- Occurs when one transaction reads data written by another uncommitted transaction.
- If the first transaction is rolled back, all dependent transactions must also roll back.

Example chain:

- T1 fails → T2 read T1 data → T2 rollback

- T3 read T2 data → T3 rollback

This is called cascading rollback.

3. Handling Cascading Rollback

- Only write_item operations are undone during rollback.
- read_item operations are logged only to detect cascading rollback.

Practical Systems

- Modern recovery methods ensure strict or cascadeless schedules.
- Therefore cascading rollback rarely occurs in practice.

22.1.6 Transaction Actions That Do Not Affect the Database

1. Non-Database Actions

Transactions may perform actions such as:

- Generating reports
- Printing messages
- Sending output to users

These actions do not directly modify the database.

2. Problem During Failure

- If a transaction fails before completion, generated reports may contain incorrect information.
- Users may take decisions based on invalid outputs.

3. Solution

- Such outputs should be generated only after the transaction commits.
- A common method is:
    - Store report generation commands as batch jobs.
    - Execute them only after the transaction commits.

If Transaction Fails

- The batch jobs are canceled, preventing incorrect outputs.

22.2 NO-UNDO / REDO Recovery Based on Deferred Update

•Deferred update means postponing all database updates on disk until the transaction successfully completes and reaches the commit point.

•During transaction execution, updates are stored only in the log file and cache buffers. After the transaction commits and the log is force-written to disk, the updates are applied to the database on disk.

•If a transaction fails before commit, the database remains unchanged; therefore UNDO is not required.

•Only REDO-type log entries are recorded in the log. These entries contain the After Image (AFIM) of the updated item. UNDO entries are unnecessary because no update occurs on disk before commit.

•This method works well mainly for short transactions with few updates. For larger transactions, updates must remain in cache buffers until commit, causing many buffers to be pinned and possibly leading to buffer space shortage.

•Deferred update protocol:

1.A transaction cannot update the database on disk before commit; modified buffers remain pinned (No-Steal policy).

2.A transaction cannot commit until all its REDO log entries are written to the log and the log buffer is force-written to disk (WAL rule).

•Since updates are written to disk only after commit, UNDO is never required. However, REDO is required if a system crash occurs after commit but before the updates are written to disk.

•In multiuser systems, concurrency control and recovery are related. When strict two-phase locking is used, write locks are held until the transaction commits, ensuring strict and serializable schedules.

•RDU_M recovery algorithm (Deferred Update with checkpoints):

oThe system keeps a commit list (transactions committed after the last checkpoint) and an active list (transactions that started but did not commit).

oDuring recovery, REDO all WRITE operations of committed transactions from the log in their original order.

oTransactions in the active list are canceled and resubmitted.

•REDO operation: From a log entry [write_item, T, X, new_value], the value of item X in the database is set to new_value (AFIM).

•Example: If checkpoint occurs at t_1 and system crashes at t_2, transaction T1 committed before the checkpoint and needs no REDO. Transactions T2 and T3 committed after the checkpoint, so their updates must be redone. Transactions T4 and T5 did not commit and are ignored or canceled.

•If a data item is updated multiple times by committed transactions after the checkpoint, only the latest update is redone during recovery by scanning the log from the end and maintaining a list of items already recovered.

•If a transaction is aborted (for example, due to deadlock detection), it is simply resubmitted, since it has not modified the database on disk.

•Advantages: No UNDO operations are needed and cascading rollback does not occur because transactions never read values written by uncommitted transactions.

•Drawbacks: Reduced concurrency due to locks held until commit and possible high buffer space requirements to store updates until commit.

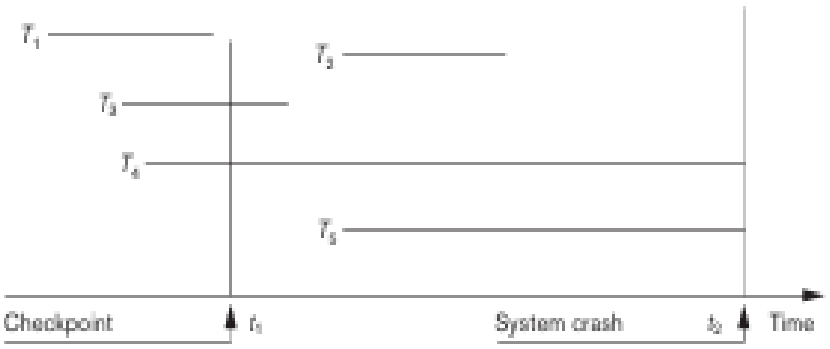


Figure 22.2
An example of a recovery timeline to illustrate the effect of checkpointing.

22.3 Recovery Techniques Based on Immediate Update

(a)

T_1
read_item(A)
read_item(D)
write_item(D)

T_2
read_item(B)
write_item(B)
read_item(D)
write_item(D)

T_3
read_item(A)
write_item(A)
read_item(C)
write_item(C)

T_4
read_item(B)
write_item(B)
read_item(A)
write_item(A)

(b)

[start_transaction, T_1]
[write_item, T_1, D, 20]
[commit, T_1]
[checkpoint]
[start_transaction, T_4]
[write_item, T_4, B, 15]
[write_item, T_4, A, 20]
[commit, T_4]
[start_transaction, T_2]
[write_item, T_2, B, 12]
[start_transaction, T_3]
[write_item, T_3, A, 30]
[write_item, T_2, D, 25]

← System crash

T_2 and T_3 are ignored because they did not reach their commit points.

T_4 is redone because its commit point is after the last system checkpoint.

Figure 22.3
An example of recovery using deferred update with concurrent transactions. (a) The READ and WRITE operations of four transactions. (b) System log at the point of crash.

- · In Immediate Update, when a transaction performs an update, the change may be written to the database on disk before the transaction commits. However, it is not necessary that every update be written immediately.
- Because updates may reach the disk before commit, the system must be able to undo the effects of failed transactions. This is done by rolling back the transaction using UNDO log entries that store the Before Image (BFIM) of the data item.
- These techniques follow the steal strategy, which allows modified buffers in main memory to be written to disk before the transaction commits.
- Immediate update methods are divided into two types: UNDO/NO-REDO: All updates of a transaction are written to disk before commit, so redo is not required. This method uses the steal/force strategy. UNDO/REDO: A transaction may commit before all updates are written to disk, so recovery requires undoing uncommitted transactions and redoing committed transactions. This method uses the steal/no-force strategy and is the most commonly used in practice.
- In multiuser systems, recovery depends on the concurrency control protocol. With strict two-phase locking, transactions cannot read or write items modified by uncommitted transactions, though deadlocks may occur, requiring transaction abort and undo.
- The recovery algorithm RIU_M (Recovery using Immediate Update – Multiuser) uses checkpoints and maintains two lists: committed transactions and active transactions since the last checkpoint.

- During recovery, the system first undoes all write operations of active (uncommitted) transactions in reverse log order, and then redoes all write operations of committed transactions in log order.
- UNDO operation: From the log entry [write_item, T, X, old_value, new_value], the value of item X is restored to old_value (BFIM).
- For efficiency, redo is applied only to the latest update of each item, and undo operations are applied at most once by maintaining lists of processed items.

22.4 Shadow Paging

- Shadow Paging is a recovery technique that does not require a log in a single-user environment. In a multiuser environment, logs may still be required for concurrency control.
- The database is considered as a collection of fixed-size disk pages (n pages). A directory with n entries is maintained, where each entry points to the corresponding database page on disk. This directory is usually kept in main memory, and all read/write operations access disk pages through it.
- When a transaction starts, the current directory is copied to create a shadow directory. The shadow directory is stored on disk and never modified, while the current directory is used for transaction operations.
- During a write operation, the modified page is written to a new disk location instead of overwriting the old page. The current directory entry is updated to point to this new page, while the shadow directory continues to point to the original page. Thus, two versions of updated pages exist:
    - o Old version → referenced by the shadow directory
    - o New version → referenced by the current directory
- If a system failure occurs during transaction execution, recovery is done by discarding the current directory and restoring the shadow directory, which brings the database back to its previous consistent state.
- When a transaction commits, the previous shadow directory is discarded, and the current directory becomes the new stable state. Since recovery requires neither UNDO nor REDO, this method is classified as a NO-UNDO / NO-REDO recovery technique.

· Disadvantages:

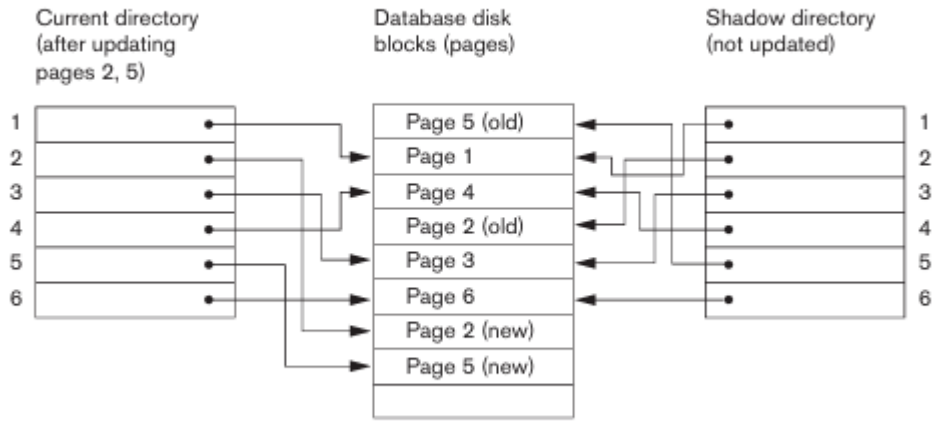
o Updated pages may be stored in different disk locations, making it difficult to keep related pages close together.

o Writing large shadow directories to disk can create overhead during commits.

o Garbage collection is required to free old pages that are no longer needed after commit.

o The switch between current and shadow directories must be atomic to maintain consistency.

Figure 22.4
An example of shadow paging.



[5]The directory is similar to the page table maintained by the operating system for each process.

22.5 The ARIES Recovery Algorithm

- ARIES (Algorithms for Recovery and Isolation Exploiting Semantics) is a widely used recovery algorithm, especially in IBM relational database systems. It follows the steal / no-force buffer management strategy
- ARIES is based on three main concepts: Write-Ahead Logging (WAL): Log records must be written to disk before the corresponding data pages are written to disk.
- Repeating History During REDO: During recovery, ARIES repeats all actions recorded in the log to rebuild the database state at the time of crash. Logging During UNDO: When an operation is undone, a compensation log record (CLR) is written so that the same undo is not repeated if another failure occurs during recovery
- The ARIES recovery process has three phases: Analysis, REDO, and UNDO.
- ANALYSIS Phase: Identifies dirty pages in the buffer and transactions that were active at the time of crash. It determines the log point from which REDO should start and rebuilds two tables:
    - o Transaction Table: Contains transaction ID, status, and the last LSN of each active transaction.
    - o Dirty Page Table: Contains page ID and the LSN of the earliest update to that page.
- ·REDO Phase :Reapplies updates from the log to ensure the database reaches the exact state at the time of crash. REDO begins from the smallest LSN in the Dirty Page Table and continues until the end of the log. ARIES checks whether a change has already been applied before redoing it to avoid unnecessary operations.
- Undo the operations of transactions that were active during the crash. The log is scanned backward, restoring data items to their before image (BFIM). For every undo action, a compensation log record (CLR) is written to the log.

- In ARIES, every log record has a Log Sequence Number (LSN), which uniquely identifies the log entry and increases sequentially. Each data page also stores the LSN of the last update applied to it, helping determine whether REDO is required.
- Log records are generated for actions such as write (update), commit, abort, undo, and end transaction. Update log records include details like page ID, offset, length, before image (BFIM), and after image (AFIM).
- Checkpointing in ARIES involves writing begin_checkpoint and end_checkpoint records to the log and storing the LSN of the begin_checkpoint in a special file. ARIES uses fuzzy checkpointing, allowing transactions to continue during checkpoint without flushing all buffers to disk.
- Example: If a crash occurs after transactions T1, T2, and T3 update pages, the analysis phase rebuilds the Transaction Table and Dirty Page Table. The REDO phase reapplies necessary updates from the smallest LSN in the Dirty Page Table. Finally, the UNDO phase reverses the actions of any active transactions (such as T3) by scanning the log backward.

(a)

Lsn	Last_lsn	Tran_id	Type	Page_id	Other_information
1	0	T_1	update	C	---
2	0	T_2	update	B	---
3	1	T_1	commit		---
4	begin checkpoint				
5	end checkpoint				
6	0	T_3	update	A	---
7	2	T_2	update	C	---
8	7	T_2	commit		---

(b)

TRANSACTION TABLE

Transaction_id	Last_lsn	Status
T_1	3	commit
T_2	2	in progress

DIRTY PAGE TABLE

Page_id	Lsn
C	1
B	2

(c)

TRANSACTION TABLE

Transaction_id	Last_lsn	Status
T_1	3	commit
T_2	8	commit
T_3	6	in progress

DIRTY PAGE TABLE

Page_id	Lsn
C	7
B	2
A	6

Figure 22.5
An example of recovery in ARIES. (a) The log at point of crash. (b) The Transaction and Dirty Page Tables at time of checkpoint. (c) The Transaction and Dirty Page Tables after the analysis phase.

.

22.6 Recovery in Multidatabase Systems

- A multidatabase transaction accesses multiple databases, which may be managed by different DBMS types (relational, object-oriented, hierarchical, etc.). Each DBMS may have its own transaction manager and recovery technique.

- To maintain atomicity across multiple databases, a two-level recovery mechanism is used:
    - Local Recovery Managers at each database
    - A Global Recovery Manager (Coordinator) that controls the overall transaction.
- The coordinator manages transactions using the Two-Phase Commit (2PC) Protocol.
- When all participating databases finish their part of the transaction, the coordinator sends a prepare for commit message. Each participant force-writes its log records to disk and replies with OK (ready to commit) or Not OK (cannot commit). If no response is received within a timeout, it is treated as Not OK.
- Phase 2 – Commit/Abort Phase:
    - If all participants reply OK, the coordinator sends a commit message, and each database records the commit in its log and permanently updates the database.
    - If any participant replies Not OK, the coordinator sends a rollback (UNDO) message, and each database undoes the local transaction operations using its log.
- The result of the 2PC protocol is that either all databases commit the transaction or none of them do, ensuring atomicity even in case of failures.
- If failure occurs before or during Phase 1, the transaction is usually rolled back. If failure occurs during Phase 2, the system can still recover and complete the commit.

22.7 Database Backup and Recovery from Catastrophic Failures

- Previous recovery techniques mainly handle non-catastrophic failures where the system log or shadow directory remains intact on disk.
- Catastrophic failures, such as disk crashes, require database backup techniques.
- A database backup involves periodically copying the entire database and log to external storage media such as magnetic tapes or other offline storage devices.
- In case of catastrophic failure, the latest backup copy is restored to disk, and the system is restarted.
- For critical applications (banking, insurance, stock markets), backups are often stored in physically separate secure locations to protect against disasters such as fires, floods, earthquakes, or terrorist attacks.
- To reduce data loss, the system log is backed up more frequently than the full database, since it is smaller and easier to copy.
- After restoring the database from the latest backup, the committed transactions recorded in the backed-up log are redone to reconstruct the database state.
- After each database backup, a new system log is started, which simplifies recovery from disk failures.